System Verification and Validation Plan for FitFolio

Team #1, Burn n' Earn
Davis Lenover
Preston Grewal
Angad Chhabra
Farid Bastoros
Zaid Alsaadi

November 1, 2026

Table 1: Revision History

Date	**Developer(s)**	**Change**
Oct 26–27, 2026	Davis Lenover	3.6 Automated Testing and Verification Tools.
Oct 29, 2026	Davis Lenover	6.2 Usability Survey Questions, with matching updates to 2.3.
Nov 1, 2026	Zaid Alsaadi	Section 3.5: Implementation Verification.

[The intention of the VnV plan is to increase confidence in the software. However, this does not mean listing every verification and validation technique that has ever been devised. The VnV plan should also be a **feasible** plan. Execution of the plan should be possible with the time and team available. If the full plan cannot be completed during the time available, it can either be modified to "fake it", or a better solution is to add a section describing what work has been completed and what work is still planned for the future. —SS]

[The VnV plan is typically started after the requirements stage, but before the design stage. This means that the sections related to unit testing cannot initially be completed. The sections will be filled in after the design stage is complete. the final version of the VnV plan should have all sections filled in. —SS]

Contents

1 Symbols, Abbreviations, and Acronyms iv

2 General Information 1
- 2.1 Summary . . . 1
- 2.2 Objectives . . . 2
- 2.3 Custom Documents . . . 4
- 2.4 Relevant Documentation . . . 4

3 Plan 5
- 3.1 Verification and Validation Team . . . 5
- 3.2 SRS Verification . . . 5
- 3.3 Design Verification . . . 5
- 3.4 Verification and Validation Plan Verification . . . 5
- 3.5 Implementation Verification . . . 6
- 3.6 Automated Testing and Verification Tools . . . 7
  - 3.6.1 Linting and Formatting . . . 8
  - 3.6.2 Unit and Integration Testing . . . 8
  - 3.6.3 Secret Scanning . . . 9
- 3.7 Software Validation . . . 9

4 System Tests 10
- 4.1 Tests for Functional Requirements . . . 10
  - 4.1.1 Accounts and Profile . . . 11
  - 4.1.2 Activity and Rewards . . . 18
  - 4.1.3 Simulated Market and Portfolio . . . 33
  - 4.1.4 Learning and Feedback . . . 55
  - 4.1.5 Social Feed . . . 61
  - 4.1.6 Reminders and Settings . . . 67
- 4.2 Tests for Nonfunctional Requirements . . . 74
  - 4.2.1 Area of Testing1 . . . 75
  - 4.2.2 Area of Testing2 . . . 75
- 4.3 Traceability Between Test Cases and Requirements . . . 75

5 Unit Test Description 76
- 5.1 Unit Testing Scope . . . 76
- 5.2 Tests for Functional Requirements . . . 76

5.2.1 Accounts and Profile 77
5.2.2 Learning and Feedback 77
5.2.3 Social Feed 78
5.3 Tests for Nonfunctional Requirements 79
5.3.1 Module ? 79
5.3.2 Module ? 79
5.4 Traceability Between Test Cases and Modules 79

6 Appendix **81**
6.1 Symbolic Parameters 81
6.2 Usability Survey Questions 81
6.2.1 Before-Study Questionnaire 82
6.2.2 Experience Log 86
6.2.3 After-Study Questionnaire 87
6.2.4 Debrief Interview 89

7 Appendix — Generative AI Use Disclosure **90**
7.1 AI Tools Used 90
7.2 How and Where Used 90
7.3 Verification of AI-Generated Content 90

8 Appendix — Reflection **91**

List of Tables

1 Revision History i
2 Parts of the usability study 82
3 Knowledge check answers 86

[Remove this section if it isn't needed —SS]

List of Figures

[Remove this section if it isn't needed —SS]

1 Symbols, Abbreviations, and Acronyms

symbol	description
T	Test

[symbols, abbreviations, or acronyms — you can simply reference the SRS (Author, 2019) tables, if appropriate —SS]

[Remove this section if it isn't needed —SS]

This document ... [provide an introductory blurb and roadmap of the Verification and Validation plan —SS]

2 General Information

This section identifies the software under test, states the objectives of verification and validation (V&V) and what is out of scope, summarizes the custom documents, and lists the documents this plan draws on.

2.1 Summary

The software being tested is FitFolio, a mobile application in which young adults earn virtual currency from the physical activity their phone records and invest it in a simulated market that follows real-world prices. FitFolio consists of a mobile app for Android and iOS, built with React Native and TypeScript, and a server built with FastAPI and a PostgreSQL database. The app reads activity from the phone's health platform (Apple HealthKit or Google Health Connect), and the server obtains asset prices from the market data provider, Finnhub. The full system and its context are described in Section 2.3 of the Software Requirements Specification (SRS; Team 1, Burn n' Earn, 2026b).

The functions of FitFolio fall into the six feature areas of SRS Section 6, which also organize the system tests in Section 4 of this plan:

- **Accounts and profile:** creating an account, signing in and out, resetting a password, editing the profile, downloading a copy of one's data, and deleting the account.
- **Activity and rewards:** reading activity records for the supported activity types, converting plausible activity into virtual currency up to a daily cap, and tracking goals, streaks, rest days and milestones.
- **Simulated market and portfolio:** buying and selling whole shares of at least 50 real-world stocks at prices that follow the real market, showing holdings and portfolio value, and practising in market scenarios that replay past periods.

- **Learning and feedback:** explaining financial and fitness concepts where they appear, giving feedback on the user's own trades and portfolio changes, and testing understanding with knowledge checks.
- **Social feed:** adding friends by username and sharing selected statistics and milestones with chosen friends.
- **Reminders and settings:** sending reminders only in the periods the user chooses, and letting the user review and withdraw health data permissions.

Because FitFolio handles health data and presents financial information that users may act on, the plan also verifies the safety and security requirements identified in the Hazard Analysis (HA; Team 1, Burn n' Earn, 2026a), which the SRS restates in Sections 6 and 7.3.

2.2 Objectives

The objectives below are the qualities the team most needs to be confident in, given the time and people available. They are listed in priority order, with the SRS goals (G) and non-functional requirements (NFR) each one comes from.

- **Correct currency, rewards, and portfolio values.** Build confidence that activity earns the right amount of virtual currency, that each activity is rewarded only once, and that balances, trades, and portfolio values are calculated correctly, including when a trade is interrupted or repeated (G4, G5, NFR1 Accuracy, NFR6 Reliability). This comes first because users judge the app by whether its numbers are right, and in an educational app a wrong number also teaches a wrong lesson.
- **Private and secure user data.** Demonstrate that private information, including health data, is available only to authenticated and authorized users, and that nothing is shared with friends unless the user chooses to share it (G3, NFR7 Security). FitFolio handles health data, so a leak would harm users and break the privacy rules it must follow (SRS, SC6).

- **Adequate usability and accessibility.** Validate through usability testing that first-time users can complete the core activity and investing tasks without help, including with enlarged text, and that participants improve on the before-and-after knowledge check (G1, G2, NFR2 Usability, NFR8 Accessibility). The results will be reported in the Usability Report.

- **Core features on Android and iOS.** Demonstrate that the core features work on both supported platforms (NFR4 Portability). Since one React Native codebase targets both, the same acceptance tests are run on each.

The following are out of scope, because the team does not control them or does not have the resources to verify them:

- **External libraries and services.** The phone's health platform (Apple HealthKit or Health Connect), the libraries that connect to it, the market data provider, and frameworks such as FastAPI, SQLAlchemy, and PostgreSQL are assumed to have been verified by their own teams. How FitFolio handles wrong, missing, late, or duplicated data from these services is still in scope (Hazard Analysis, CA4).

- **Accuracy of recorded activity and market prices.** FitFolio rewards the activity the health platform records and values portfolios at the provider's prices. Whether a phone counts steps correctly, or whether a quoted price matches the exchange, cannot be checked by the team. The team verifies how FitFolio uses these values, but not the values themselves.

- **Load and scalability testing.** No requirement covers large numbers of users, and during the project the app will be used only by the team, the supervisor, and usability test participants. Performance is checked only as stated in NFR5.

- **Attacks from a compromised phone or hosting provider.** Security testing covers FitFolio's own authentication and authorization. A rooted or jailbroken phone and the hosting provider's physical security are outside the team's control (Hazard Analysis, CA3 and CA5). No external penetration test is planned.

- **Long-term effects on users.** Whether FitFolio changes users' fitness habits or financial decisions over months cannot be measured within the project. Learning is measured only by the short-term knowledge check in G1.

2.3 Custom Documents

FitFolio has two custom documents (CDs), one in each term, as listed in Section 4 of the Problem Statement and Goals. They are the project's extras, and both feed directly into validation.

Design Thinking Report (Fall). This report explores what young adults want to track about their fitness and what they hope to learn about investing. Its main evidence is two exploratory interviews with potential users, held on October 14, 2026, and referred to as P01 and P02. These interviews are the source of most of the stakeholder requirements in SRS Section 3 and of Personas 1 and 2 in SRS Section 4. For V&V, the report provides the needs that validation checks the product against. It also exposes a gap that validation must close: neither interviewee was a beginner at investing, so the needs of beginners, represented by Persona 3, come from the Problem Statement rather than from users. The usability tests in this plan therefore recruit mainly participants who are beginners at investing (Section 3.7).

Usability Report (Winter). This report documents usability testing of the working application's core loop: earning currency from steps, buying shares, and following the portfolio's value. The usability tests specified in this plan are the ones the report will run and record: the participant profile, the tasks, and the questions are defined here (Section 3.7 and Appendix 6.2), and the report presents their results. These tests produce the evidence for NFR2 (Usability), NFR8 (Accessibility), and goals G1 (at least 70% of participants improve their score on a before-and-after knowledge check) and G2 (at least 80% of first-time participants complete the core tasks without outside help) from SRS Section 2.2. Because the tests need a working application, they take place after the Rev 0 demonstration.

2.4 Relevant Documentation

[Reference relevant documentation. This will definitely include your SRS and your other project documents (design documents, like MG, MIS, etc). You can include these even before they are written, since by the time the project is done, they will be written. You can create BibTeX entries for your documents and within those entries include a hyperlink to the documents. —SS]

Author (2019)

[Don't just list the other documents. You should explain why they are relevant and how they relate to your VnV efforts. —SS]

3 Plan

[Introduce this section. You can provide a roadmap of the sections to come. —SS]

3.1 Verification and Validation Team

[Your teammates. Maybe your supervisor. You should do more than list names. You should say what each person's role is for the project's verification. A table is a good way to summarize this information. —SS]

3.2 SRS Verification

[List any approaches you intend to use for SRS verification. This may include ad hoc feedback from reviewers, like your classmates (like your primary reviewer), or you may plan for something more rigorous/systematic. —SS]

[If you have a supervisor for the project, you shouldn't just say they will read over the SRS. You should explain your structured approach to the review. Will you have a meeting? What will you present? What questions will you ask? Will you give them instructions for a task-based inspection? Will you use your issue tracker? —SS]

[Maybe create an SRS checklist? —SS]

3.3 Design Verification

[Plans for design verification —SS]

[The review will include reviews by your classmates —SS]

[Create a checklists? —SS]

3.4 Verification and Validation Plan Verification

[The verification and validation plan is an artifact that should also be verified. Techniques for this include review and mutation testing. —SS]

[The review will include reviews by your classmates —SS]

[Create a checklists? —SS]

3.5 Implementation Verification

The team will check FitFolio's implementation through testing, code review, and automated checks. These activities will verify that the code meets the Software Requirements Specification (SRS), including the safety and security requirements from the Hazard Analysis. The review and merge process will follow the Development Plan's Workflow Plan and Definition of Done.

Testing: The team will run the system tests in Section 4 and implement the unit tests described in Section 5 as features are developed. System tests will check the application against its functional and nonfunctional requirements. Unit tests will check individual parts of the code, while integration tests will check how components work together, including backend operations that use the database. The preliminary unit tests in Section 5 will be extended and mapped to modules once the Module Interface Specification is complete.

For each feature, the author will add or update relevant tests and explain their purpose in the pull request. Tests will cover normal behaviour, boundary values, and invalid inputs where applicable. Following the priorities in Section 2.2, testing will pay particular attention to reward and portfolio calculations, repeated activity records, interrupted or repeated trades, and access to private user data. Expected numerical results will be derived from the SRS rules and symbolic parameters, independently of the implementation being tested. When fixing a defect, the author will add a test where practical to detect the same problem if it occurs again.

Code Review: Before merging a change, at least one team member other than the main author will inspect the code and its tests. If all team members contributed to the feature, everyone must approve it. The reviewer will normally work in a related area. Changes affecting the whole team will be submitted for team review, following the Development Plan.

Reviewers will use the Source Code Checklist and the Development Plan's Coding Standard to check that:

- The changed behaviour matches the relevant requirements and, once available, the module interfaces.
- Calculations, input checks, error handling, and access restrictions handle the cases affected by the change.
- Tests cover the changed behaviour and relevant failure cases, with expected results justified by the specification.
- Names, comments, code structure, and constants make the code understandable and consistent with the coding standard.

Reviewers will leave findings on the pull request. The author will address them, and the reviewer will check the revisions before approving. Defects found outside a pull request will be recorded as GitHub issues and linked to their fixes.

Automated Checks and Records: Section 3.6 describes the tools used to support verification. Ruff lints and formats the backend's Python code, and gitleaks scans the repository for secrets. The frontend's TypeScript code will be linted with ESLint and formatted with Prettier. Backend tests run with pytest, and frontend unit tests will use Jest. Reviewers can use coverage reports to check that a pull request tests the code it adds. No minimum coverage is enforced at this stage.

Changes must receive the approvals required by the Development Plan and pass all CI checks before merging. Pull request discussions, test output, and linked issues will record the checks performed and problems found. For manual system tests, the tester will record the test identifier, platform, observed result, and any defect found. Results and unresolved limitations will be summarized in the VnV Report.

3.6 Automated Testing and Verification Tools

Ruff lints and formats the backend's Python code, and pytest runs its unit and integration tests. For the frontend, ESLint and Prettier lint and format the TypeScript code, and Jest runs its unit tests. gitleaks scans the whole repository for secrets. Each runs in GitHub Actions on every pull request and every push to `main`, and the team's workflow requires every check to pass before a pull request is merged (Development Plan, Workflow Plan). The reasons for choosing each tool are given in the Development Plan's Expected Technology section.

3.6.1 Linting and Formatting

Ruff is used to lint and format the backend's Python code, in place of the separate Black and Flake8 tools. It is configured in `src/backend/pyproject.toml` and enforces the team's coding standard (Development Plan, Coding Standard), including the PEP 8 line length of 79 characters. The enabled rule sets are:

- **E, W:** pycodestyle errors and warnings (PEP 8 style).
- **F:** pyflakes, such as unused imports and undefined names.
- **I:** import order.
- **B:** flake8-bugbear, which flags likely bugs such as mutable default arguments.
- **UP:** pyupgrade, which flags syntax that has a newer equivalent.
- **N:** PEP 8 naming conventions.
- **SIM:** flake8-simplify, which flags code that can be written more simply.

The `Lint (Ruff)` job in `.github/workflows/backend-ci.yml` runs `ruff check` and `ruff format --check` on both the backend source and its tests in `test/backend`. The job fails if any rule is broken or any file is not formatted, so unformatted code cannot reach `main`.

The frontend's TypeScript code will be linted with ESLint, which catches common mistakes, and formatted with Prettier, configured to use four spaces for indentation to match the coding standard. Both will run in CI on every pull request and fail the check if any rule is broken or any file is not formatted, in the same way as Ruff.

3.6.2 Unit and Integration Testing

pytest is used for the backend's automated tests, which live in `test/backend`. The `Test (pytest + PostgreSQL)` job in `.github/workflows/backend-ci.yml` starts a PostgreSQL 18 container, applies the database migrations with Alembic, and then runs the full test suite. Tests that read or write data therefore run against a real database with the same schema the app uses, rather than a mock. The CI database is created fresh for each run and holds no real user data.

Code coverage is measured with pytest-cov. Each CI run prints a coverage report that lists, for every module in the backend's `app` package, the percentage of lines executed and the line numbers that were missed. Reviewers can use this report to check that a pull request tests the code it adds. No minimum coverage is enforced at this stage. Coverage results will be summarized in the VnV Report.

The frontend's unit tests will use Jest, which works directly with React Native, and will run in CI alongside the backend tests. Jest has built-in coverage reporting, so frontend coverage will be reported on each run and summarized in the VnV Report in the same way.

3.6.3 Secret Scanning

The repository is public, so an API key, password, or token is exposed as soon as it is pushed. Secrets are kept in a local `.env` file, which Git ignores, and gitleaks checks for any that slip into a commit at two points:

- **Before each commit:** a pre-commit hook, configured in `.pre-commit-config.yaml`, scans the staged changes and stops the commit if they contain a secret. Each member sets it up once per clone, as described in `CONTRIBUTING.md`.
- **In CI:** the `Secret scan` workflow in `.github/workflows/secret-scan.yml` scans the commits in every pull request and every push to `main`. This catches a secret even if the hook was skipped or never set up.

Both checks use gitleaks 8.30.1, so they apply the same detection rules. Any secret found is redacted in the output, so a CI run does not expose it again. A secret that reaches the repository is revoked and replaced, since removing it from the history does not make it private again.

3.7 Software Validation

[If there is any external data that can be used for validation, you should point to it here. If there are no plans for validation, you should state that here. —SS]

[You might want to use review sessions with the stakeholder to check that the requirements document captures the right requirements. Maybe task based inspection? —SS]

[For those capstone teams with an external supervisor, the Rev 0 demo should be used as an opportunity to validate the requirements. You should plan on demonstrating your project to your supervisor shortly after the scheduled Rev 0 demo. The feedback from your supervisor will be very useful for improving your project. —SS]

[For teams without an external supervisor, user testing can serve the same purpose as a Rev 0 demo for the supervisor. —SS]

[This section might reference back to the SRS verification section. —SS]

4 System Tests

[There should be text between all headings, even if it is just a roadmap of the contents of the subsections. —SS]

4.1 Tests for Functional Requirements

The tests in this section are grouped into the same six areas as the functional requirements (FRs) in SRS Section 6: Accounts and Profile, Activity and Rewards, Simulated Market and Portfolio, Learning and Feedback, Social Feed, and Reminders and Settings. Each test is derived from the fit criterion of the FRs it covers, so an FR is verified when all of its tests pass. Every FR is covered by at least one test, and the traceability table that ends the System Tests section maps each test back to its FRs.

Each test follows the format of the template, with these conventions:

- **Test ID:** `test-`*area*`-`*n*, where *area* is `acct`, `act`, `mkt`, `learn`, `social`, or `remind`, matching the SRS labels for each area, and *n* numbers the tests within the area.
- **Requirements:** the FRs the test covers, by their SRS number.

- **Control:** *Automatic* tests run with pytest against the backend's API, or with Jest for the frontend, on every pull request (see Automated Testing and Verification Tools). *Manual* tests are run by a tester on an Android or iOS test device, mostly to check what the user sees.

Automated tests send activity records and market prices to the backend directly, in place of the health platform and the market data provider, so every input and expected value is known. The accuracy of those services is out of scope (see Objectives). Manual tests record activity on a test device through the phone's health platform, so they also check that FitFolio reads it correctly.

Tests that depend on the calendar day run in a fixed time zone, with the backend's clock set to known dates, so that daily totals, streaks, and rest days can be tested without waiting for real days to pass. Expected values use the symbolic constants in SRS Appendix C, such as `MAX_DAILY_CURRENCY`, with their current values given in parentheses. If a constant changes, the expected outputs of the tests that use it change with it.

4.1.1 Accounts and Profile

These tests cover the accounts and profile requirements in SRS Section 6.1 (FR1 to FR11). Automatic tests create their own test accounts through the backend's API, with email addresses in the reserved `.test` domain, so no message can reach a real person. Tests of password reset replace the email service with a test mailbox that records every message sent to it, so the test can read the reset link and check who it was sent to. Passwords in the inputs are at least `MIN_PASSWORD_LENGTH` (8) characters long, so that the password rule in the Hazard Analysis (SEC-38) does not reject them for a reason unrelated to the test.

1. **test-acct-1: Creating an account and signing in**

   Requirements: FR1, FR4

   Control: Automatic

   Initial State: No account uses the email address `sam@fitfolio.test` or the username `sam_test`.

Input: A sign-up request with the email address `sam@fitfolio.test`, the username `sam_test`, the password `Walk2026run`, and the age confirmed. Then two sign-in requests with that email address: one with the password `Walk2026run`, and one with `Walk2026ruN`.

Output: The sign-up succeeds and an account with that email address exists. The first sign-in succeeds and returns a session. The second is refused and returns no session.

Test Case Derivation: The fit criterion of FR1 is that a user who enters an unused email address and a valid password has an account and can sign in with them, which the first sign-in shows. FR4 signs in a user with the correct password and refuses a wrong one. The wrong password differs from the correct one only in the case of its last letter, so a comparison that ignored case would let it through.

How test will be performed: A pytest test sends the three requests to the backend's API and checks each response.

2. **test-acct-2: No account without confirming age**

   Requirements: FR2

   Control: Automatic and Manual

   Initial State: The email addresses `young1@fitfolio.test` and `young2@fitfolio.test` are not used by any account.

   Input: Two sign-up requests to the backend, with otherwise valid details:

   - for `young1@fitfolio.test`, with the age confirmation set to false;
   - for `young2@fitfolio.test`, with the age confirmation left out of the request.

   Then, on a test device, the tester fills in the sign-up screen for `young3@fitfolio.test` without confirming their age.

   Output: Both requests are refused, and signing in with either email address fails because no account exists. On the device, no account is created for `young3@fitfolio.test`, and the screen shows that the age confirmation is required.

Test Case Derivation: FR2 (Hazard Analysis, SEC-37) creates no account until the user confirms they are at least `MIN_USER_AGE` (18) years old. Each case uses its own email address, so that an account existing afterwards shows which case failed. The two requests check the backend: a confirmation set to false must be refused, and so must a request with no confirmation at all, since a backend that only checked for false would treat a missing value as confirmed. The backend is tested directly because a modified app could skip the check on the sign-up screen (Hazard Analysis, H43). The device case checks that the sign-up screen enforces the confirmation too and tells the user why. It needs only one case, because the screen always sends a value.

How test will be performed: A pytest test sends the two requests and then tries to sign in with each address. The tester then tries the sign-up screen on a test device and records whether an account was created.

3. **test-acct-3: Unique usernames**

   Requirements: FR3

   Control: Automatic

   Initial State: An account with the username `sam_test` exists, as created in test-acct-1. No account uses the email address `jordan@fitfolio.test`.

   Input: A sign-up request for `jordan@fitfolio.test` with the username `sam_test`, followed by a second request with the same details except the username `jordan_test`.

   Output: The first request is refused with a message asking the user to choose a different username, and no account is created. The second request succeeds.

   Test Case Derivation: FR3 asks a user who chooses a username already in use to choose a different one. The second request shows that the refusal was caused by the username alone, since every other detail is the same.

   How test will be performed: A pytest test sends both requests and checks each response and which accounts exist afterwards.

4. **test-acct-4: Signing out**

   Requirements: FR5

   Control: Manual

   Initial State: A test user with a balance of 250 and the username `sam_test`, signed in on a test device.

   Input: The tester opens the home screen and notes the balance shown, signs out, closes FitFolio completely, and opens it again.

   Output: After reopening, FitFolio shows the sign-in screen. Neither the balance of 250 nor the username `sam_test` appears on any screen reachable without signing in.

   Test Case Derivation: The fit criterion of FR5 is that after signing out, opening the app shows the sign-in screen and none of the user's data. Closing the app completely checks that the sign-out is kept on the phone and is not just a change of screen. The balance and username are known values, so the tester can check that neither is shown.

   How test will be performed: The tester follows the input steps on the test device and records each screen shown after reopening.

5. **test-acct-5: Resetting a forgotten password**

   Requirements: FR6

   Control: Automatic

   Initial State: An account for `sam@fitfolio.test` with the password `Walk2026run`, and a second account for `jordan@fitfolio.test`. The email service is replaced by a test mailbox.

   Input: A password reset request for `sam@fitfolio.test`. The test opens the link in the email the mailbox receives and sets the new password `Swim2027lap`. It then signs in with each password.

   Output: The mailbox receives exactly one email, addressed to `sam@fitfolio.test` and containing a reset link. After the reset, signing in with `Walk2026run` is refused and signing in with `Swim2027lap` succeeds. The account for `jordan@fitfolio.test` receives no email and its password is unchanged.

Test Case Derivation: FR6 (Hazard Analysis, SEC-40) sends the link to the account's own email address, and its fit criterion is that the user can then sign in only with the new password. Checking the old password catches a reset that adds a new password without removing the old one. The second account checks that a reset for one account does not reach or change another.

How test will be performed: A pytest test sends the reset request, reads the email from the test mailbox, follows the link through the API, and sends the sign-in requests.

6. **test-acct-6: Editing the profile**

   Requirements: FR7

   Control: Manual

   Initial State: A test user signed in on a test device, with a height of 170 cm in their profile.

   Input: The tester changes the height to 175 cm and saves it, closes FitFolio completely, opens it again, and opens the profile.

   Output: The profile shows a height of 175 cm.

   Test Case Derivation: The fit criterion of FR7 is that the user sees the new value the next time they open their profile. Closing the app between saving and reopening checks that the change was stored, and not only kept on the screen.

   How test will be performed: The tester follows the input steps on the test device and records the height shown.

7. **test-acct-7: Using every feature without height or weight**

   Requirements: FR8

   Control: Automatic

   Initial State: A test user with no height or weight in their profile, a balance of 0, the health notice and simulated market notice acknowledged, and one accepted friend. Test asset A is priced at 20.00, its market is open, and its price is within `QUOTE_STALENESS_LIMIT` (15 minutes).

Input: A daily total of 5,000 steps for day d. The user then buys 1 share of A, opens the concept explanation of *Stock*, completes the knowledge check on *Stock* with every answer correct, and shares their streak with their friend.

Output: The balance increases by 50, then decreases by 20.00 to 30.00 when the share is bought. The explanation opens, the knowledge check returns a result, and the shared streak appears in the friend's social feed. No request is refused for missing height or weight.

Test Case Derivation: The fit criterion of FR8 names four features, earning, trading, learning and sharing, and the test uses each one with both values empty. 5,000 steps earn $\lfloor 5000/100 \rfloor = 50$ (SRS Section 6.7), and one share at 20.00 costs 20.00. A feature that wrongly required height or weight would refuse its request or fail to calculate a result.

How test will be performed: A pytest test sends each request through the API and checks each response, the balance, and the friend's feed.

8. **test-acct-8: Downloading a copy of one's data**

   Requirements: FR9

   Control: Automatic

   Initial State: A test user with the username `sam_test`, an age of 25, a height of 170 cm, and a weight of 68 kg. They have daily totals of 4,000 steps on day $d-1$ and 6,000 steps on day d, one purchase of 2 shares of asset A at 15.00 each, a balance of 70.00, and one shared update of their streak. A second user, their friend `jordan_test`, has their own activity, holdings and shared updates.

   Input: A request from `sam_test` for a copy of their data.

   Output: The user receives a JavaScript Object Notation (JSON) file containing their username, age, height and weight, both daily totals, the balance of 70.00, the holding of 2 shares of A, the purchase with its price of 15.00, and the shared update. It contains none of `jordan_test`'s activity, holdings or shared updates.

   Test Case Derivation: The fit criterion of FR9 lists the profile, activity totals, balance, holdings, trades and shared updates, and each has a known value here so the file can be checked field by field. The balance is $100 - 2 \cdot 15.00 = 70.00$ for a user who earned 100 from 10,000 steps. The SRS sets JSON as the format for data that FitFolio exchanges (SRS Section 1.2). The friend's data checks that the copy contains only the requester's data.

How test will be performed: A pytest test sets up both users through the API, requests the copy, parses the file, and compares each field with the values above.

9. **test-acct-9: Confirming account deletion**

   Requirements: FR10

   Control: Manual

   Initial State: A test user signed in on a test device.

   Input: The tester starts account deletion, reads the confirmation message, and chooses to cancel. The tester then signs out and signs in again.

   Output: Before anything is deleted, a message states that deleting the account cannot be undone. After cancelling, the user can sign in again and their data is unchanged.

   Test Case Derivation: The fit criterion of FR10 is that the warning is shown and the account still exists if the user cancels. Signing in again checks that cancelling did not disable the account or start its deletion.

   How test will be performed: The tester follows the input steps on the test device and records the message and the result of signing in.

10. **test-acct-10: Deleting an account**

    Requirements: FR11

    Control: Automatic

    Initial State: Test users `sam_test` and `jordan_test` are friends. `sam_test` has shared one update with `jordan_test` and holds a valid session.

    Input: `sam_test` requests account deletion and confirms it.

Output: Signing in as `sam_test` is refused. A request made with `sam_test`'s earlier session is refused. `jordan_test`'s social feed no longer contains `sam_test`'s update, and `sam_test` is no longer in their friend list.

Test Case Derivation: The fit criterion of FR11 is that the user can no longer sign in and that none of their data is shown to them or to their friends. The earlier session checks that deletion also ends existing sessions, since otherwise data would still be shown to the user. The friend's feed and friend list check what friends can see. Removal from the server and its backups within `DATA_DELETION_PERIOD` (30 days) is a Hazard Analysis requirement (SEC-18) in SRS Section 7.3 and is verified with the nonfunctional requirements.

How test will be performed: A pytest test sends the deletion and confirmation through the API, then tries to sign in, sends a request with the old session, and reads `jordan_test`'s feed and friend list.

4.1.2 Activity and Rewards

These tests cover the activity and rewards requirements in SRS Section 6.2 (FR12 to FR27). Expected currency comes from the daily reward R_d in SRS Section 6.7, with the values in SRS Appendix C: one unit of currency per 100 steps or wheelchair pushes, one per minute of cycling, swimming, or another workout, and at most `MAX_DAILY_CURRENCY` (300) per day.

The server calculates rewards only from the daily activity totals the app sends (Hazard Analysis, SEC-24), and stores no start or end times (SRS Data Dictionary, Activity Record). Tests of the reward rules therefore run against the backend with daily totals. Tests that need the time of each activity, such as assigning it to a calendar day or checking a workout's length, run against the frontend with Jest. The plausibility limits are input constraints on activity records, so each limit is tested at its value and just above it.

1. **test-act-1: Earning for each activity type**

   Requirements: FR12, FR13

   Control: Automatic

   Initial State: Six test users, each with a balance of 0, the health notice acknowledged, no activity goal, and no activity on day d.

Input: For day d, users 1 to 5 each have a total for one supported type: 2,550 steps, 1,099 wheelchair pushes, 20 minutes of cycling, 15 minutes of swimming, and 30 minutes of strength training, which counts as another workout. User 6 has all five totals.

Output: The balances of users 1 to 5 increase by 25, 10, 20, 15, and 30. The balance of user 6 increases by 100.

Test Case Derivation: Users 1 to 5 each have a single type, so each balance increase is that type's reward alone, and a type that earns nothing or earns at the wrong rate shows up on its own (FR12). By R_d, each type earns $\lfloor x_{t,d}/u_t \rfloor$, so 2,550 steps earn 25 and 1,099 pushes earn 10, which also checks that partial units are rounded down. Each minute of a workout earns 1. User 6 checks that R_d adds up the rewards of all types (FR13): $25 + 10 + 20 + 15 + 30 = 100$. This is below the daily cap of 300, so the cap cannot hide a wrong sum.

How test will be performed: A pytest test sends each user's daily totals to the backend, then reads each balance through the API.

2. **test-act-2: Daily earning cap**

   Requirements: FR18

   Control: Automatic

   Initial State: Three test users, each with a balance of 0, the health notice acknowledged, and no activity on day d.

   Input: For day d, the users have totals of 29,900, 30,000, and 40,000 steps.

   Output: The balances increase by 299, 300, and 300.

   Test Case Derivation: $R_d = \min(300, \lfloor x_{t,d}/100 \rfloor)$ gives 299, 300, and $\min(300, 400) = 300$. The three totals check the cap just below, at, and above `MAX_DAILY_CURRENCY` (300).

   How test will be performed: As in test-act-1.

3. **test-act-3: Activity counts toward the day in the user's time zone**

   Requirements: FR18

Control: Automatic

Initial State: A test user whose phone is set to the America/Toronto time zone (UTC−5 in January).

Input: Two step records: 20,000 steps ending at 23:30 local time on January 14, 2027 (04:30 UTC on January 15), and 20,000 steps ending at 00:30 local time on January 15, 2027.

Output: The app's daily totals are 20,000 steps for January 14 and 20,000 steps for January 15.

Test Case Derivation: FR18 counts each activity toward the calendar day in the user's time zone at the time of the activity. Each day then earns $\lfloor 20000/100 \rfloor = 200$. Grouping the records by UTC day instead would put 40,000 steps on January 15, which earns only the cap of 300, so the test detects the wrong grouping.

How test will be performed: A Jest test passes the two records, with their timestamps and the user's time zone, to the app's code that builds daily totals, and checks the totals it returns.

4. **test-act-4: Day with no activity data**

   Requirements: FR16

   Control: Automatic

   Initial State: A test user with a balance of 100, the health notice acknowledged, and no activity goal.

   Input: The backend receives no activity totals for day d, and its clock is then moved to day $d + 1$.

   Output: Day d shows a total of 0 for every supported type, and the balance is still 100.

   Test Case Derivation: FR16 treats a day with no data as a day with zero activity. With $x_{t,d} = 0$ for every type, $R_d = 0$.

   How test will be performed: A pytest test reads the totals for day d and the balance through the API.

5. **test-act-5: Daily step and push limits**

Requirements: FR17

Control: Automatic

Initial State: Four test users, each with a balance of 0, the health notice acknowledged, no activity goal, a streak of 6, and no activity on day d.

Input: For day d, the users have totals of 150,000 steps, 150,001 steps, 100,000 wheelchair pushes, and 100,001 wheelchair pushes.

Output: The totals at each limit (150,000 steps and 100,000 pushes) are counted: those users' balances increase by 300, the totals appear in their activity trends, their streaks become 7, and they reach the 7-day streak milestone. The totals over each limit earn nothing, do not appear in the users' trends, reset their streaks to 0, reach no milestone, and are marked as not counted in the API's response.

Test Case Derivation: Each user has one total, so each limit is checked on its own. The limits are `MAX_DAILY_STEPS` (150,000) and `MAX_DAILY_PUSHES` (100,000), and FR17 leaves out a record only when it exceeds its limit, so a total at the limit is counted and a total one above it is not. Testing both sides of each limit catches a check that uses $\geq$ instead of $>$. The totals at the limit earn more than the cap, so $R_d = 300$. With no goal set, any counted activity extends the streak (SRS Section 6.7), so a streak of 6 becomes 7. FR25 shows each milestone in `MILESTONES` (streaks of 7, 30, and 100 days, and 1,000, 10,000, and 100,000 currency earned in total), so a streak of 7 reaches the first streak milestone. A total over the limit is left out of $x_{t,d}$, so $R_d = 0$ and day d does not count toward the streak, which resets it to 0. Starting from a streak of 6 means that a wrongly counted total would reach the 7-day milestone, so the test also checks that a total that is not counted reaches no milestone.

How test will be performed: A pytest test sends each user's totals, then reads the balance, trends, streak, and each total's status through the API.

6. **test-act-6: Workout length limit**

Requirements: FR17

Control: Automatic

Initial State: A test user whose phone has no activity recorded on days d and $d + 1$.

Input: A cycling record on day d lasting exactly 12 hours, and a cycling record on day $d + 1$ lasting 12 hours and 1 minute.

Output: The app counts the first workout, giving a cycling total of 720 minutes for day d. It leaves the second workout out, giving a cycling total of 0 for day $d + 1$, and marks it as not counted.

Test Case Derivation: `MAX_WORKOUT_DURATION` is 12 hours, and FR17 leaves out only a workout longer than it, so the two records check the limit at its value and one minute above it. They are on separate days because two workouts this long cannot fit in one day. Checking a single workout's length needs its start and end times, which only the phone stores, so the app applies this limit before it sends the daily total.

How test will be performed: A Jest test passes the records to the app's code that builds daily totals, and checks the total and each record's status.

7. **test-act-7: Showing a record as not counted**

   Requirements: FR17

   Control: Manual

   Initial State: A test user signed in on a test device, who acknowledged the health notice at least two days ago.

   Input: The tester enters a cycling workout of 12 hours and 1 minute for the previous day in the phone's health platform, then opens FitFolio.

   Output: FitFolio shows the workout as not counted, and the user's balance, activity trends, and streak are the same as before the workout was entered.

   Test Case Derivation: FR17 requires the user to be shown that the record was not counted. test-act-5 and test-act-6 check the underlying values automatically, and this test checks the whole path on a real device. The workout is entered for the previous day so that all of it fits within one finished day, whatever time the test is run. The user acknowledged the notice before that day, because activity from before the user's first day would not count for a different reason (SRS Section 6.7).

How test will be performed: The tester notes the balance, trends, and streak, enters the workout, then checks the same values and the workout's label in FitFolio.

8. **test-act-8: No currency before the health notice is acknowledged**

   Requirements: FR14

   Control: Automatic

   Initial State: A new test user with a balance of 0 and no activity goal, who has not acknowledged the health notice or consented to FitFolio reading their health data.

   Input: The backend receives a total of 5,000 steps for day d. On day $d + 1$, the user records 3,000 steps, then acknowledges the notice and consents. The backend then receives the totals for days d and $d + 1$.

   Output: The balance is 0 at the end of day d and 30 at the end of day $d + 1$. At the end of day $d + 1$, the streak is 1 and the activity trends show only day $d + 1$. The app reads no health data and asks for no consent until the notice is acknowledged, and its first sync sends no total for a day before the user's first day.

   Test Case Derivation: FR14 allows no currency until the notice is acknowledged, even if a daily total reaches the server, so day d earns nothing at first. The app reads no health data before then, so that total can only come from an app bug or a modified app, and the test checks that the server does not rely on the app to enforce the notice. The user's first day is the calendar day they acknowledge the notice, $d + 1$ (SRS Section 6.7), and SEC-20 does not reward records from before it, so day d still earns nothing when its total is sent again. The app should not send that total, so this also checks that the server does not rely on the app to leave it out. Day $d + 1$ earns $\lfloor 3000/100 \rfloor = 30$. Its steps come before the acknowledgement, which checks that the cut-off is the calendar day and not the moment of acknowledgement. The two days have different totals so that the final balance shows which days were rewarded: 80 would mean day d was rewarded too. With no goal set, the streak would be 2 if day d counted (SRS Section 6.7).

How test will be performed: A pytest test sends day d's total, moves the backend's clock to day $d + 1$, sends the acknowledgement, the consent, and then both days' totals, and reads the balance at the end of each day, then the streak and trends. A Jest test replaces the health platform with a test version that records every read, as in test-remind-7, and holds records from the previous day. It starts the app for a new user, checks that no read and no consent request happen before the notice is acknowledged, and checks that the first sync sends no total for the previous day.

9. **test-act-9: Health notice shown before the consent request**

   Requirements: FR14

   Control: Manual

   Initial State: A new account signed in on a test device for the first time. The phone's health platform already holds workouts from earlier days and no activity from the current day.

   Input: The tester opens FitFolio, acknowledges the notice, and consents to FitFolio reading their health data.

   Output: Before asking for consent, FitFolio shows a notice stating that it does not give medical advice and that users should consult a health professional before starting a new exercise program. The consent request does not appear until the tester acknowledges the notice. After the tester consents and the first sync finishes, the balance is still 0 and the earlier workouts do not appear in the activity trends.

   Test Case Derivation: This is the notice and the order FR14 requires. A new user's health platform may already hold many workouts, so the test starts with some. They are from before the user's first day (SRS Section 6.7), so they earn nothing and are not shown (FR27). There is no activity from the current day, so any currency after the first sync could only come from those workouts. test-act-8 sends totals to the backend directly, and this test checks the same rule on a real device, with real records in the health platform.

How test will be performed: The tester compares the notice's text with FR14, checks that no consent request appears before acknowledging it, and checks the balance and activity trends after the first sync.

10. **test-act-10: Rewards without opening the app**

    Requirements: FR15

    Control: Manual

    Initial State: Two test users on separate test devices, each with the health notice acknowledged and no activity recorded for the current day.

    Input: Both testers record a workout of between 10 and 11 minutes with the phone's own workout tracking. Tester A keeps FitFolio closed for the whole workout. Tester B opens FitFolio during it. Both open FitFolio after the workout.

    Output: Both balances increase by 10.

    Test Case Derivation: At one unit per minute, a workout of between 10 and 11 minutes earns 10, since partial minutes are rounded down. FR15 requires the same reward whether or not the user interacts with FitFolio during the activity. The two testers do the workout together, so the only difference between them is whether they open FitFolio during it.

    How test will be performed: Each tester notes their balance before the workout and compares it with the balance after opening FitFolio.

11. **test-act-11: Rewards ignore body weight and calories**

    Requirements: FR26

    Control: Automatic

    Initial State: A test user with a body weight of 70 kg, a balance of 0, the health notice acknowledged, and no activity on days d to $d+2$.

    Input: The user has a total of 30 minutes of cycling on each of days d, $d+1$, and $d+2$. Before day $d+1$, the weight is changed to 90 kg, and before day $d+2$, it is removed.

    Output: The balance increases by 30 on each day, to 90 at the end of day $d+2$.

Test Case Derivation: The fit criterion of FR26 changes and removes the user's weight, so this test does the same, with the same activity on each day. Only the weight differs between the days, so if it affected the reward, at least one day would earn a different amount. R_d depends only on activity totals and earning rates, so 30 minutes of cycling earns 30 on each day. The activity record has no calorie field (SRS Data Dictionary), so calories cannot reach the reward calculation. Goals are pairs of an activity type and an amount (SRS Section 6.7), and `MILESTONES` are streaks of 7, 30, and 100 days and 1,000, 10,000, and 100,000 currency earned in total (SRS Appendix C), so neither uses weight or calories either.

How test will be performed: A pytest test sends each day's total and the weight changes, moving the backend's clock between days, and reads the balance at the end of each day.

12. **test-act-12: Goal changes apply from the next day**

    Requirements: FR19

    Control: Automatic

    Initial State: A test user with a daily goal of 10,000 steps, a streak of 3, the health notice acknowledged, and no activity on days d and $d+1$.

    Input: On day d, the user lowers the goal to 5,000 steps. The user then has a total of 6,000 steps on each of days d and $d+1$.

    Output: The goal in effect is 10,000 steps on day d and 5,000 steps on day $d+1$. The streak is 0 at the end of day d and 1 at the end of day $d+1$.

    Test Case Derivation: FR19 applies a goal change from the next day, so the goal in effect on day d is still 10,000. Day d's 6,000 steps miss it, so day d does not count toward the streak (SRS Section 6.7) and the streak resets to 0. On day $d+1$, 6,000 steps meet the new goal of 5,000, so the streak becomes 1. The daily total is between the old and new goals, so whether each day counts shows which goal was in effect. The goal is lowered, not raised, because applying changes from the next day is what stops a user from lowering a goal to keep the current day's streak alive (Hazard Analysis, SAF-4). The streak starts at 3 so that a change applied on the same day would give a streak of 4 instead of 0.

How test will be performed: A pytest test sends the goal change and then the day's total, moves the backend's clock to day $d+1$, sends that day's total, and reads the goal in effect and the streak for each day through the API.

13. **test-act-13: Earning without a goal**

    Requirements: FR20

    Control: Automatic

    Initial State: Two test users, each with a balance of 0, the health notice acknowledged, and no activity on day d. User 1 has no activity goal. User 2 has a daily goal of 3,000 steps.

    Input: For day d, both users have a total of 4,000 steps.

    Output: Both balances increase by 40.

    Test Case Derivation: User 1 is the case in the fit criterion of FR20: a user with no goal who records activity earns currency. User 2 has the same steps and meets their goal, so comparing the two shows whether a user without a goal earns less than one who meets a goal. R_d has no goal term (SRS Section 6.7), so both earn $\lfloor 4000/100 \rfloor = 40$. A reward that paid extra for meeting a goal would push users to set goals they did not choose (Hazard Analysis, SAF-3), and this test would detect one.

    How test will be performed: As in test-act-1.

14. **test-act-14: No automatic goal changes**

    Requirements: FR21

    Control: Automatic

    Initial State: A test user with a daily goal of 5,000 steps, the health notice acknowledged, and no activity on days d to $d+6$.

    Input: The user has a total of 10,000 steps on each of days d to $d+6$, and does not change the goal. The backend's clock is then moved to day $d+7$.

    Output: The goal in effect on day $d+7$ is still 5,000 steps.

Test Case Derivation: The fit criterion of FR21 has the user exceed their goal on `GOAL_REVIEW_PERIOD` (7) consecutive days, which are days d to $d+6$. Each day's total is twice the goal, so the goal is clearly exceeded throughout, which is when an automatic increase would be most likely. A goal change applies from the next day (FR19), so a change made at the end of the period would first take effect on day $d+7$, which is when the goal is read. The user confirms no change, so any goal other than 5,000 fails the test. A change the user does confirm is tested in test-act-12.

How test will be performed: A pytest test sends each day's total, moving the backend's clock between days, and reads the goal in effect on day $d+7$ through the API.

15. **test-act-15: Streak with a goal**

Requirements: FR22, FR23

Control: Automatic

Initial State: A test user with a daily goal of 5,000 steps, a streak of 0, the health notice acknowledged, no rest days, and no activity on days d to $d+4$.

Input: The user has totals of 6,000, 5,000, and 6,000 steps on days d to $d+2$. Day $d+3$ is marked as a rest day and has no activity. Day $d+4$ is not a rest day and has 4,999 steps.

Output: The streak at the end of days d to $d+4$ is 1, 2, 3, 4, and 0.

Test Case Derivation: The days follow the fit criterion of FR22: three days that meet the goal give a streak of 3, a rest day then gives 4, and a missed day then gives 0. By SRS Section 6.7, a day counts toward the streak when the user meets every goal, so $x_{t,d} \geq g$, or when it is a rest day. Day $d+1$ has exactly the goal and day $d+4$ is one step below it, so the goal is checked at its value and just below, which catches a check that uses $>$ instead of $\geq$. Day $d+4$ also has activity, which shows that with a goal set, recording activity is not enough to keep the streak. As in the fit criterion of FR23, the rest day has no activity, so the streak continues because of the rest day alone.

How test will be performed: A pytest test sends each day's total or rest day, moving the backend's clock between days, and reads the streak at the end of each day through the API.

16. **test-act-16: Streak with more than one goal**

    Requirements: FR22

    Control: Automatic

    Initial State: A test user with daily goals of 5,000 steps and 20 minutes of cycling, a streak of 0, the health notice acknowledged, and no activity on days d and $d + 1$.

    Input: The user has 8,000 steps and 20 minutes of cycling on day d, and 8,000 steps and 19 minutes of cycling on day $d + 1$.

    Output: The streak is 1 at the end of day d and 0 at the end of day $d + 1$.

    Test Case Derivation: SRS Section 6.7 counts a day toward the streak only when the user meets every goal in effect. Day d meets both goals. On day $d + 1$, the step goal is met but cycling is one minute short, so the streak resets. The two days differ only in cycling, so the change in the streak comes from the cycling goal alone. This catches a streak that counts a day when any one goal is met.

    How test will be performed: As in test-act-15.

17. **test-act-17: Streak without a goal**

    Requirements: FR22

    Control: Automatic

    Initial State: A test user with no activity goal, a streak of 0, the health notice acknowledged, and no activity on days d to $d + 2$.

    Input: The user has 50 steps on day d and 1 minute of swimming on day $d + 1$. The backend receives no activity totals for day $d + 2$.

    Output: The streak at the end of days d to $d + 2$ is 1, 2, and 0.

    Test Case Derivation: The fit criterion of FR22 increases the streak of a user with no goal on each day they record any activity, and SRS Section 6.7 counts such a day when the sum of the day's totals is above

0. The 50 steps earn no currency, since $\lfloor 50/100 \rfloor = 0$, so day d shows that the streak depends on activity, not on currency earned. Day $d+1$ uses a different activity type to show that any supported type counts, not only steps. Day $d+2$ has no activity, which counts as zero (FR16), so the streak resets.

How test will be performed: As in test-act-15.

18. **test-act-18: Rest day limit**

    Requirements: FR23

    Control: Automatic

    Initial State: A test user with no rest days marked, in the week from Monday, January 4, 2027, to Sunday, January 10, 2027.

    Input: The user asks to mark rest days on Tuesday, January 5, Thursday, January 7, Sunday, January 10, and Monday, January 11, 2027. Each request is made on the day it is for.

    Output: The requests for January 5, 7, and 11 are accepted, and the request for January 10 is refused. The user's rest days are January 5, 7, and 11.

    Test Case Derivation: `REST_DAYS_PER_PERIOD` is 2 and `STREAK_PERIOD` is a calendar week from Monday to Sunday in the user's time zone (SRS Appendix C). January 5 and 7 use the two rest days of the week of January 4, so the request for January 10 is a third in the same week and is refused. January 10 is a Sunday, so the request also checks that Sunday ends the week that began on Monday. If weeks started on Sunday, it would be the first rest day of a new week and would be accepted. January 11 is the next Monday, so it starts a new week and is accepted. A rolling 7-day window would still contain January 5 and 7 and would refuse it. Each request is made on its own day so that the test does not depend on whether rest days can be marked for other days.

    How test will be performed: A pytest test moves the backend's clock to each date, sends the rest day request, checks whether it is accepted, and then reads the user's rest days through the API.

19. **test-act-19: No penalty for a missed goal**

    Requirements: FR24

    Control: Automatic

    Initial State: A test user with a daily goal of 10,000 steps, a streak of 5, a balance of 500, 3 shares of a stock, the health notice acknowledged, and no activity on day d. The user has one friend and has shared their streak of 5 with them.

    Input: The user has a total of 4,000 steps on day d and does not mark it as a rest day. The backend's clock is then moved to day $d + 1$.

    Output: At the end of day d, the balance is 540 and the user holds 3 shares. At the start of day $d+1$, the streak is 0, the balance is still 540, the user still holds 3 shares, and the friend's social feed is the same as at the start of day d.

    Test Case Derivation: The user misses their goal on day d and does not take a rest day, so the streak ends (SRS Section 6.7), which is the case in the fit criterion of FR24. The streak starts at 5 so that there is a streak to end. The balance and holdings start above 0 so that a reduction would show. The day's steps still earn $\lfloor 4000/100 \rfloor = 40$, as they would with no goal set (Hazard Analysis, SAF-2), so the balance at the end of day d is 540, and FR24 requires the same balance and holdings at the start of day $d + 1$. Holdings are compared as numbers of shares, so a change in the stock's price cannot affect the result. The user has already shared their streak, so there is an update that could reveal the ended streak. The user shares nothing on day d, so the friend's feed should not change. Comparing the whole feed catches a new update about the missed goal or the ended streak, and also the shared streak being changed to 0.

    How test will be performed: A pytest test sends day d's total, reads the balance and holdings, moves the backend's clock to day $d + 1$, and reads the streak, balance, holdings, and the friend's feed through the API.

20. **test-act-20: Streak milestones**

Requirements: FR25

Control: Automatic

Initial State: Three test users with no activity goal, the health notice acknowledged, no currency earned, and streaks of 6, 29, and 99 days.

Input: Each user has 50 steps on each of days d and $d+1$. User 1 then has no activity on day $d+2$ and 50 steps on each of days $d+3$ to $d+9$.

Output: On day d, the users are shown the 7-day, 30-day, and 100-day streak milestones. No user is shown a milestone on day $d+1$. User 1's streak is 7 again at the end of day $d+9$, and they are not shown a milestone.

Test Case Derivation: The streak values in `MILESTONES` are 7, 30, and 100 days (SRS Appendix C). Each user's streak is one below a value, so with no goal set, one day of activity reaches it (SRS Section 6.7). There is one user for each value, which catches a wrong value in the list. On day $d+1$, each streak is one past its milestone, which checks that the milestone is shown once and not again while the streak is above it. User 1's streak then resets and reaches 7 again, but the fit criterion of FR25 shows a milestone only when the streak first reaches it, so it is not shown again. The 50 steps earn no currency (as in test-act-17), so no currency milestone can be reached during the test.

How test will be performed: A pytest test sends each day's totals, moving the backend's clock between days, and reads the milestones to show each user through the API at the end of each day.

21. **test-act-21: Currency milestones**

Requirements: FR25

Control: Automatic

Initial State: Three test users with no activity goal, a streak of 0, and the health notice acknowledged, who have earned 990, 9,990, and 99,990 currency in total. Each has a balance of 10, having spent the rest on simulated trades.

Input: Each user has a total of 900 steps on day d, 100 steps on day $d+1$, and 100 steps on day $d+2$.

Output: No user is shown a milestone on day d. On day $d+1$, the users are shown the 1,000, 10,000, and 100,000 currency milestones. No user is shown a milestone on day $d+2$.

Test Case Derivation: The currency values in `MILESTONES` are 1,000, 10,000, and 100,000 currency earned in total (SRS Appendix C), with one user for each. Day d earns 9, which brings each total to one below its value, and day $d+1$ earns 1, which brings it to the value exactly, so each value is checked just below and at it. Day $d+2$ checks that the milestone is shown once. When the milestones are reached, each balance is only $10+9+1=20$, so a milestone based on the balance instead of the total earned would not be shown. The streaks reach only 3, so no streak milestone can be reached during the test.

How test will be performed: As in test-act-20.

22. **test-act-22: Activity trends**

Requirements: FR27

Control: Automatic

Initial State: A test user with the health notice acknowledged and no activity on days d to $d+13$.

Input: On each day $d+i$, for i from 0 to 13, the user has $1000+i$ steps, $2000+i$ wheelchair pushes, $10+i$ minutes of cycling, $30+i$ minutes of swimming, and $50+i$ minutes of another workout.

Output: The user's activity trends show each of the five totals for each of the 14 days, equal to the totals sent.

Test Case Derivation: The 14 days come from the fit criterion of FR27. Each activity type has its own range of totals, and each day's total is one more than the day before, so all 70 totals are different. A total shown under the wrong day or the wrong type is therefore detected. All of the totals are within the plausibility limits (FR17), so all of them are counted and shown.

How test will be performed: A pytest test sends each day's totals, moving the backend's clock between days, then reads the user's trends through the API and compares them with the totals sent.

4.1.3 Simulated Market and Portfolio

These tests cover the simulated market and portfolio requirements in SRS Section 6.3 (FR28 to FR49). Expected balances, holdings, and portfolio values come from the definitions of trades and portfolio value in SRS Section 6.7, and are compared to within `VALUE_TOLERANCE` (0.01) (NFR1). All prices are in `QUOTE_CURRENCY` (US dollars).

Automated tests replace the market data provider, Finnhub (SRS Appendix C), with a test provider that returns the prices each test chooses, so every expected price is known. Each test also sets whether the asset's market is open, so tests that depend on market hours can run at any time of day. Price age is measured with the backend's clock. Manual tests use the real provider, so they also check that FitFolio shows real prices. The exceptions are manual tests that need a price change the real market is unlikely to give during the test, such as a large change in portfolio value. These run the release build against a test backend that uses the test provider.

1. **test-mkt-1: Number of tradable stocks**

   Requirements: FR28

   Control: Manual

   Initial State: A new test user, who holds no assets, signed in on a test device with the release build of FitFolio.

   Input: The tester opens the live market and counts the stocks that can be bought, leaving out exchange-traded funds (ETFs).

   Output: The count is at least `MIN_TRADABLE_STOCKS` (50).

   Test Case Derivation: `MIN_TRADABLE_STOCKS` is 50 (SRS Appendix C), from Problem Statement Goal 4. The fit criterion of FR28 counts only stocks, so ETFs are left out. The asset list is set by administrators on the server (FR30), not in the app's code, so the test runs against the list that users will actually see.

   How test will be performed: The tester scrolls through the live market list, counts each stock with an option to buy it, and records the count.

2. **test-mkt-2: Prices in one currency**

Requirements: FR29

Control: Automatic

Initial State: Three test assets, each with a current price of 50.00. The test provider quotes two of them, both listed in the United States, in US dollars, and the third, listed in Canada, in Canadian dollars. A test user with a balance of 1,000 who has acknowledged the simulated market notice.

Input: An administrator adds the three assets to the live market. While the market is open, the user then reads the list of tradable assets and asks to buy one share of each of the three assets.

Output: The list contains only the two assets quoted in US dollars, each with its price in US dollars. Their purchases are accepted, and the purchase of the asset quoted in Canadian dollars is refused.

Test Case Derivation: FR29 offers only assets quoted in US dollars, the value of `QUOTE_CURRENCY` (SRS Appendix C). An asset listed in Canada is the case the Hazard Analysis gives for a price in another currency (SEC-27), so the third asset is one that the test expects not to be offered. Without it, the test could not fail. Each share costs 50.00, well below the balance, and every price is current, so a refused purchase can only be due to the currency. Checking both the list and a purchase catches an asset that is hidden from the list but can still be traded. The third asset may be refused when it is added or left out of the list afterwards, and the test passes either way, since in both cases it is not offered.

How test will be performed: A pytest test sets the test provider's prices, adds the assets as an administrator, reads the list of tradable assets through the API, sends each purchase, and checks which are accepted.

3. **test-mkt-3: Adding and removing an asset**

Requirements: FR30, FR31

Control: Automatic

Initial State: A test asset that is not in the live market, with a price of 40.00 from the test provider throughout the test. A test user with a balance of 1,000 who holds no assets and has acknowledged the simulated market notice.

Input: While the market is open, an administrator adds the asset to the live market, and the user buys 2 shares. The administrator then removes the asset, and the user asks to buy 1 share.

Output: The first purchase is accepted, leaving a balance of 920 and 2 shares. After the removal, the asset is not in the list of tradable assets and the user holds no shares of it. The balance is 1,000, the trade history shows a sale of 2 shares at 40.00, and the user is told what was sold and at what price. The purchase after the removal is refused, and the balance is still 1,000.

Test Case Derivation: The steps follow the fit criteria of FR30 and FR31: an added asset can be traded, and removing it sells the user's shares at its most recent price and stops it from being bought. By the trade rules in SRS Section 6.7, the purchase gives $1000 - 2 \cdot 40 = 920$ and the sale gives $920 + 2 \cdot 40 = 1000$. The user holds 2 shares so that a sale of only one would show. The price stays at 40.00 so that each change in the balance comes only from a trade. The market is open and the price is current here, and test-mkt-27 covers a closed market, an old price, and queued trades.

How test will be performed: A pytest test adds and removes the asset as an administrator, sends each purchase, and reads the balance, holdings, trade history, and list of tradable assets through the API after each step.

4. **test-mkt-4: Changing the asset list without an app update**

   Requirements: FR30, FR31

   Control: Manual

   Initial State: A test user with a balance of 1,000 who holds no assets and has acknowledged the simulated market notice, signed in on a test device with the release build of FitFolio. A stock listed in the United States that is not in the live market.

   Input: While the US market is open, an administrator adds the stock to the live market, and the tester buys one share of it. The administrator then removes the stock, and the tester checks FitFolio.

Output: After it is added, the stock appears in the live market and can be bought. After it is removed, it no longer appears in the live market and cannot be bought, the holding is gone, the balance has increased by the price of the sale, and FitFolio tells the tester what was sold and at what price. The installed version of FitFolio is the same at the end of the test as at the start.

Test Case Derivation: The fit criterion of FR30 requires that no app update happen between the changes, which only a test on an installed app can show. test-mkt-3 checks the server's rules, and this test checks that the installed app follows the change. The tester may close and reopen FitFolio, but not update it. The test uses the real provider, so it is run while the US market is open. Otherwise, the purchase would be queued until the market opens (FR40).

How test will be performed: The tester notes the installed version of FitFolio. An administrator adds and removes the stock with the server's administrator tools. After the addition, the tester buys a share. After the removal, the tester checks the live market, their holdings and balance, and the message about the sale. At the end, the tester checks the installed version again.

5. **test-mkt-5: Prices from the latest quote**

   Requirements: FR32

   Control: Automatic

   Initial State: An asset in the live market whose last price from the test provider is 40.00. A test user with a balance of 1,000 who holds no assets and has acknowledged the simulated market notice.

   Input: While the market is open, the test provider sends a new price of 41.50. The user reads the price and buys one share. The test provider then sends a new price of 42.25, and the user reads the price and sells the share.

   Output: The prices read are 41.50 and 42.25. After the purchase, the balance is 958.50, and after the sale, it is 1,000.75, with no shares.

   Test Case Derivation: FR32 prices each asset at the latest price from the provider, and its fit criterion covers both the price shown and the price used for trades. The price changes before each trade, so a trade or display that used an old or cached price would give a different result. By the trade rules in SRS Section 6.7, the purchase gives $1000 - 41.50 = 958.50$ and the sale gives $958.50 + 42.25 = 1000.75$. Using the old price for the purchase would leave 960 after it, and using the old price for the sale would leave 1,000.00 at the end.

How test will be performed: A pytest test sets the test provider's prices, reads each price through the API, sends the two trades, and reads the balance and holdings after each one.

6. **test-mkt-6: Price time and the delayed marker**

   Requirements: FR33

   Control: Automatic

   Initial State: An asset in the live market with one price, obtained from the test provider at time p. The provider sends no later price during the test.

   Input: While the asset's market is open, the price is read at 15 minutes after p and at 15 minutes and 1 second after p. The market then closes, and the price is read 2 days after p.

   Output: All three reads show the price with time p. Only the read at 15 minutes and 1 second after p is marked as delayed.

   Test Case Derivation: `QUOTE_STALENESS_LIMIT` is 15 minutes (SRS Appendix C), from Problem Statement Goal 4. FR33 marks a price as delayed only when it is older than the limit, so a price exactly 15 minutes old is not delayed and one a second older is. The backend's clock is set by the test, so a difference of one second can be tested exactly. The read while the market is closed is far older than the limit, which checks that the marker depends on the market being open, as the fit criterion of FR33 requires. This is the case of a price from the last close being shown overnight or on a weekend. No new price is obtained during the test, so every read returns the stored price. Reading a stored price does not change the time it was obtained (SRS Data Dictionary, Market Data), so all three reads show time p. A time that changed on each read would hide the delay.

How test will be performed: A pytest test sets the test provider's price, moves the backend's clock to each time, and reads the price, its time, and whether it is delayed through the API. A Jest test then renders the app's price display with each of the three responses and checks that it shows the time and shows the "Delayed" marker only for the delayed price.

7. **test-mkt-7: No trade before the simulated market notice is acknowledged**

   Requirements: FR34

   Control: Automatic

   Initial State: A new test user with a balance of 1,000 who holds no assets, has acknowledged the health notice, and has not acknowledged the simulated market notice. An asset in the live market with a current price of 50.00.

   Input: While the market is open, the backend receives a request from the user to buy one share. The user then acknowledges the simulated market notice, and the backend receives a second request to buy one share.

   Output: The first request is refused with a response saying that the notice must be acknowledged, and the balance is still 1,000 with no shares. The second request is accepted, leaving a balance of 950 and one share.

   Test Case Derivation: FR34 refuses a trade request that reaches the server before the user acknowledges the notice. The app shows the notice before it sends a trade (test-mkt-8), so the first request can only come from an app bug or a modified app, and the test checks that the server does not rely on the app to enforce the notice. The balance covers the share and the price is current, so the notice is the only reason for the first request to be refused, and the second request, made after the acknowledgement, is accepted. The user has already acknowledged the health notice (FR14), which catches a system that treats the two notices as one.

   How test will be performed: A pytest test sends the first purchase, checks the response, balance, and holdings, sends the acknowledgement and the second purchase, and reads the balance and holdings again through the API.

8. **test-mkt-8: Simulated market notice shown before the first trade**

   Requirements: FR34

   Control: Manual

   Initial State: A new test account on a test device that has acknowledged the health notice, has earned enough currency from recorded activity to buy one share of a stock in the live market, and has not made a trade.

   Input: While the US market is open, the tester tries to buy one share of the stock.

   Output: Before the trade completes, FitFolio shows a notice stating that the market is simulated, that virtual currency and holdings have no real-world value, and that nothing in FitFolio is financial advice. The balance and holdings do not change until the tester acknowledges the notice, and the purchase then completes.

   Test Case Derivation: The three statements are the ones listed in the fit criterion of FR34. test-mkt-7 checks the server's rule, and this test checks what the user sees on a real device. The test uses the real provider, so it is run while the US market is open, so that the trade completes instead of being queued (FR40).

   How test will be performed: The tester compares the notice's text with the three statements, and checks the balance and holdings before and after acknowledging it.

9. **test-mkt-9: Buying whole shares**

   Requirements: FR35

   Control: Automatic

   Initial State: A test user with a balance of 1,000 who holds no assets and has acknowledged the simulated market notice. An asset in the live market with a current price of 40.00.

Input: While the market is open, the user asks to buy 0, 0.5, and -1 shares, then 1 share, then 3 shares.

Output: The requests for 0, 0.5, and -1 shares are refused, and the balance is still 1,000. After the purchase of 1 share, the balance is 960 and the user holds 1 share. After the purchase of 3 shares, the balance is 840 and the user holds 4.

Test Case Derivation: FR35 lets the user buy only whole shares, and SRS Section 6.7 accepts a purchase of n shares only when n is a positive integer. The number of shares is an input, so the test uses three kinds of value that are not allowed, a fraction (0.5), zero, and a negative number, and the smallest value that is (1). A purchase of -1 shares that was accepted would work as a sale and add 40 to the balance. The purchase of 1 share is the fit criterion of FR35: the balance falls by the price shown, $1000 - 40 = 960$, and the holding rises by one. The purchase of 3 shares checks that the cost is $n \cdot p_a = 3 \cdot 40 = 120$, not the price of one share.

How test will be performed: A pytest test sends each purchase and reads the balance and holdings through the API after each one.

10. **test-mkt-10: Selling whole shares**

    Requirements: FR36

    Control: Automatic

    Initial State: A test user with a balance of 100 who holds 5 shares of an asset and has acknowledged the simulated market notice. The asset is in the live market with a current price of 40.00.

    Input: While the market is open, the user asks to sell 0, 0.5, and -1 shares, then 1 share, then 2 shares.

    Output: The requests for 0, 0.5, and -1 shares are refused, and the balance and holding are unchanged. After the sale of 1 share, the balance is 140 and the user holds 4 shares. After the sale of 2 shares, the balance is 220 and the user holds 2.

    Test Case Derivation: SRS Section 6.7 accepts a sale of n shares only when n is a positive integer, so the number of shares is tested as in test-mkt-9. A sale of -1 shares that was accepted would work as a purchase. The sale of 1 share is the fit criterion of FR36: the balance rises by the price shown, $100 + 40 = 140$, and the holding falls by one. The sale of 2 shares checks that the proceeds are $n \cdot p_a = 80$. The user keeps some shares, so the limit on selling more shares than are held is left to test-mkt-11.

How test will be performed: As in test-mkt-9, with sales.

11. **test-mkt-11: Trades limited by balance and holdings**

    Requirements: FR37

    Control: Automatic

    Initial State: Two test users who hold no assets and have acknowledged the simulated market notice, with balances of 120.00 and 119.99. An asset in the live market with a current price of 40.00.

    Input: While the market is open, each user asks to buy 3 shares. User 1 then asks to sell 4 shares, and then 3 shares.

    Output: User 1's purchase is accepted, leaving a balance of 0.00 and 3 shares. User 2's purchase is refused, and their balance is still 119.99 with no shares. User 1's sale of 4 shares is refused and changes neither the balance nor the holding. Their sale of 3 shares is accepted, leaving a balance of 120.00 and no shares.

    Test Case Derivation: SRS Section 6.7 accepts a purchase only when $n \cdot p_a \leq B$ and a sale only when $n \leq h_a$, and FR37 rejects any trade outside these limits. Each limit is tested at its value and just past it. Three shares cost $3 \cdot 40 = 120$, which equals user 1's balance and is more than user 2's by 0.01, the smallest amount a cost can exceed a balance by. User 1 then holds 3 shares, so a sale of 3 is at the limit and a sale of 4 is one share over it. This catches a check that uses $<$ instead of $\leq$. The refused trades are checked for changes to both the balance and the holding, since FR37 requires that a rejected trade change neither.

    How test will be performed: A pytest test sends each trade and reads the balance and holdings of each user through the API after each one.

12. **test-mkt-12: Interrupted trades**

Requirements: FR37

Control: Automatic

Initial State: A test user with a balance of 1,000 who holds no assets and has acknowledged the simulated market notice. An asset in the live market with a current price of 40.00.

Input: While the market is open, the user asks to buy 1 share, and the backend is made to fail after it updates one of the balance and the holding but before it updates the other. The user then makes a new request to buy 1 share, with no failure. Last, the user asks to sell 1 share, and the backend is made to fail in the same way.

Output: After the first purchase, the balance is 1,000 and the user holds no shares. After the second purchase, the balance is 960 and the user holds 1 share. After the sale, the balance is still 960 and the user still holds 1 share.

Test Case Derivation: FR37 records each trade completely or not at all, so a trade that fails partway leaves both the balance and the holding as they were. The failure is placed between the two updates because that is the only point where a trade could be left half done, with the currency spent but no share received, or the reverse. The order of the two updates depends on the backend's design, so the failure comes after whichever update is first. The completed purchase is the second case in the fit criterion of FR37, a trade that changes both values: $1000 - 40 = 960$ and one share. Purchases and sales update the values in opposite directions, so both are interrupted.

How test will be performed: A pytest test replaces the backend's step for the later of the two updates with one that raises an error, sends the trade, and then reads the balance and holdings through the API. It restores the step before the second purchase and replaces it again before the sale.

13. **test-mkt-13: Duplicate trade requests**

    Requirements: FR38

    Control: Automatic

Initial State: A test user with a balance of 1,000 who holds no assets and has acknowledged the simulated market notice. An asset in the live market with a current price of 40.00.

Input: While the market is open, the user sends request R to buy 1 share, and its response is dropped. The app sends R again. The user then sends a new request R' to buy 1 share.

Output: After both copies of R, the balance is 960, the user holds 1 share, and their trade history has one trade. Both responses to R refer to that trade. After R', the balance is 920, the user holds 2 shares, and the history has two trades.

Test Case Derivation: The dropped response is the case in the fit criterion of FR38: the app cannot tell whether the trade happened, so it retries. A retry keeps the original request's identifier (SRS Data Dictionary, Trade), so FR38 records the trade only once. R' asks for the same asset and number of shares as R, but is a new request, so it is recorded. This catches a system that drops a trade because it matches an earlier one, which would stop a user who really wants to buy the same thing twice. The test is also run with both copies of R sent at the same time, which catches a check for duplicates that two requests can both pass before either is recorded.

How test will be performed: A pytest test sends R, discards the response, sends R again, then sends R', and reads the balance, holdings, and trade history through the API after each request. A second run sends both copies of R at once from two threads.

14. **test-mkt-14: No trades on out-of-date prices**

    Requirements: FR39

    Control: Automatic

    Initial State: A test user with a balance of 1,000 who holds no assets and has acknowledged the simulated market notice. An asset with a price of 40.00, obtained from the test provider at time p. The asset's market is open throughout the test.

    Input: At 15 minutes after p, the user asks to buy 1 share. At 15 minutes and 1 second after p, the user asks to buy 1 share and to sell 1 share. The test provider then sends a new price of 41.00, and the user asks to buy 1 share again.

Output: The first purchase is executed, leaving a balance of 960 and 1 share. The purchase and sale at 15 minutes and 1 second are refused with a response saying that the price is out of date, and the balance and holding do not change. The last purchase is executed at 41.00, leaving a balance of 919 and 2 shares.

Test Case Derivation: FR39 uses `QUOTE_STALENESS_LIMIT` (15 minutes), the same limit that FR33 uses to mark a price as delayed (Hazard Analysis, SEC-3), so the test uses the same times as test-mkt-6: a trade at the limit proceeds, and one a second past it is refused. FR39 applies to every trade, so both a purchase and a sale are refused. The new price shows that the refusals were due only to the price's age, and that trading resumes once a current price is obtained, at that price (FR32): $960 - 41 = 919$.

How test will be performed: A pytest test sets the test provider's prices, moves the backend's clock to each time, sends each trade, and reads the response, balance, and holdings through the API. A Jest test renders the app's response to a refused trade and checks that the user is told the price is out of date.

15. **test-mkt-15: Trades queued while the market is closed**

    Requirements: FR40

    Control: Automatic

    Initial State: A test user with a balance of 1,000 who holds no assets and has acknowledged the simulated market notice. An asset whose market is closed, with a last price of 40.00 from the previous close.

    Input: While the market is closed, the user asks to buy 1 share. The market then opens, and the test provider sends a price of 42.00 and then a price of 43.00. The user's balance and holdings are then read.

    Output: The response gives the trade the status queued, with a notice that it will run at the opening price. After the market opens, the trade is executed at 42.00, leaving a balance of 958 and 1 share.

Test Case Derivation: FR40 queues a trade requested while the market is closed and executes it at the first price obtained after the market opens. The last price before the open, the first price after it, and a later price are all different, giving balances of 960, 958, and 957, so the balance shows which price was used. The later price arrives before the balance is read, which catches a backend that runs queued trades at whatever price is current when it reaches them, not at the first price after the open. The price from the previous close is older than `QUOTE_STALENESS_LIMIT`, so the test also checks that FR39 does not refuse the trade while the market is closed.

How test will be performed: A pytest test sends the purchase while the market is closed, checks the response, opens the market, sends the two prices, and reads the balance, holdings, and trade through the API. A Jest test renders the app's response to the queued trade and checks that it shows the trade as queued with the notice.

16. **test-mkt-16: Queued trades that can no longer be covered**

    Requirements: FR40

    Control: Automatic

    Initial State: An asset whose market is closed, with a last price of 40.00. Three test users who have acknowledged the simulated market notice. Users 1 and 2 hold no assets and have balances of 100.00 and 99.99. User 3 holds 1 share of the asset and has a balance of 0.

    Input: While the market is closed, users 1 and 2 each ask to buy 2 shares, and user 3 makes two separate requests to sell 1 share. The market then opens, and the first price is 50.00.

    Output: User 1's purchase is executed, leaving a balance of 0.00 and 2 shares. User 2's purchase is rejected, user 2 is told, and their balance is still 99.99 with no shares. One of user 3's sales is executed, leaving a balance of 50.00 and no shares. The other is not executed, and user 3 is told.

    Test Case Derivation: When they are requested, all the trades can be covered at the last price: 2 shares cost $2 \cdot 40 = 80$, which both balances cover, and user 3 holds the share that each sale needs. At the opening price, 2 shares cost $2 \cdot 50 = 100$, which equals user 1's balance and is

0.01 more than user 2's, so the limit is tested at its value and just past it, as in test-mkt-11. The two sales together need 2 shares and user 3 holds 1, so only one can be executed. The second sale may be refused when it is requested or rejected when the market opens. Either way it is not executed and the user is told, so the test passes in both cases.

How test will be performed: As in test-mkt-15, with the status of each user's trades also read through the API.

17. **test-mkt-17: Queued trade with real prices**

    Requirements: FR40

    Control: Manual

    Initial State: A test user who has acknowledged the simulated market notice and has enough currency to buy 1 share of a widely traded stock in the live market, signed in on a test device with the release build of FitFolio.

    Input: While the US market is closed, the tester notes the last price FitFolio shows for the stock and asks to buy 1 share. The tester then keeps FitFolio closed until at least 30 minutes after the market opens at 9:30 a.m. Eastern time on the next trading day, and opens it.

    Output: When it is requested, the trade is shown as queued, with a notice that it will run at the opening price. When FitFolio is opened, the trade is shown as executed, at a time after 9:30 a.m. Eastern time and before the app was opened, and at a price different from the one noted. The balance has fallen by that price.

    Test Case Derivation: This test exists because the automated tests of FR40, test-mkt-15 and test-mkt-16, cannot show how the queue behaves with real prices. In those tests, the test provider sends the first price after the open when the test chooses. With the real provider, that price arrives whenever the provider publishes it, and a quote just after the open may still be the last price from before it. FR40 requires the first price obtained after the market opens, so a trade run at that older price is the failure this test looks for. It compares the two prices because a trade that executed after 9:30 a.m. at the older price would otherwise pass. A widely traded stock is used because its price is very likely to change overnight, so a matching price points to this failure. The tester waits 30 minutes, which is longer than `QUOTE_STALENESS_LIMIT` (15 minutes), so an execution time before the app was opened shows that the backend ran the trade by itself. The test also checks, on a real device, that the user sees the queued trade and its notice.

How test will be performed: The tester requests the trade in the evening and notes the stock's last price and the balance. After waiting, the tester checks the trade's status, time, and price in FitFolio, and the new balance. If the execution price matches the noted price, the tester repeats the test with another stock, and a second match fails the test.

18. **test-mkt-18: Holdings**

    Requirements: FR41

    Control: Automatic

    Initial State: A test user who holds 3 shares of asset A and 7 shares of asset B, both in the live market.

    Input: The user's holdings are read and shown.

    Output: The holdings show asset A with 3 shares and asset B with 7 shares, and no other assets.

    Test Case Derivation: The fit criterion of FR41 has a user who owns shares of two assets see both holdings, each with the number of shares held. The two holdings have different numbers of shares, so a number shown under the wrong asset is detected.

    How test will be performed: A pytest test reads the holdings through the API. A Jest test renders the app's holdings screen with that response and checks the assets and numbers of shares shown.

19. **test-mkt-19: Portfolio value**

    Requirements: FR42

    Control: Automatic

    Initial State: A test user with a balance of 100 who holds 3 shares of asset A and 7 shares of asset B, both in the live market. The last prices from the test provider are 41.50 for A and 12.34 for B.

Input: The test provider sends a new price of 42.25 for A. The user's portfolio value is then read and shown.

Output: The portfolio value is 213.13.

Test Case Derivation: By SRS Section 6.7, $V = \sum_a h_a \cdot p_a$ with the most recent price of each asset, so $V = 3 \cdot 42.25 + 7 \cdot 12.34 = 126.75 + 86.38 = 213.13$, a value calculated by hand as the fit criterion of FR42 requires. The setup is chosen so that two common mistakes each give a value more than `VALUE_TOLERANCE` (0.01) away from it. Using A's old price would give 210.88. Adding the balance, which V does not include, would give 313.13.

How test will be performed: A pytest test sets the test provider's prices and reads the portfolio value through the API. A Jest test renders the app's portfolio screen with that response and checks the value shown. Both values are compared with 213.13 to within `VALUE_TOLERANCE`.

20. **test-mkt-20: Simulated labels**

    Requirements: FR43

    Control: Manual

    Initial State: A test user signed in on a test device with the release build of FitFolio, connected to a test backend that uses the test provider. The user bought 2 shares of asset A and 1 share of asset B at 40.00 each. Both prices were 40.00 on the previous day, when the user last viewed their portfolio, and are now 50.00 for A and 30.00 for B. Portfolio notifications are on, and the test is run within the user's reminder period. The user also has a market scenario in progress, continued until one of its holdings has risen and another has fallen.

    Input: The tester opens every screen in FitFolio's navigation, and each notification received during the test.

    Output: Every return and portfolio value shown on a screen or in a notification has a "Simulated" label next to it.

    Test Case Derivation: The fit criterion of FR43 covers every screen that shows a return or portfolio value, so the setup makes every kind of value appear. A has a gain of 25% and B a loss of 25%, so returns of both signs are shown. The portfolio value has risen from $3 \cdot 40 = 120$ to $2 \cdot 50 + 30 =$

130, a change of 8.3%, which is more than `FEEDBACK_THRESHOLD` (5%). This brings up the feedback on the change in value (FR52) and the portfolio notification (FR75), which both show values. The scenario adds its own screens. Notifications are included because FR43 covers every displayed return and value, not only those on screens. The screens come from the app's navigation, so a screen added later is also added to the test.

How test will be performed: The tester lists the screens in the app's navigation, opens each one, and records any return or portfolio value that has no "Simulated" label next to it.

21. **test-mkt-21: Gains and losses without colour**

    Requirements: FR44

    Control: Manual

    Initial State: As in test-mkt-20, with the phone's greyscale display setting turned on.

    Input: The tester opens the same screens and notifications as in test-mkt-20.

    Output: The tester can tell every gain from every loss, each shown with a sign or a text label.

    Test Case Derivation: The fit criterion of FR44 shows the display in greyscale, which removes the difference between a gain shown only in green and a loss shown only in red. A's gain and B's loss are both 25%, so their size cannot tell them apart, and only a sign or label can. The setup from test-mkt-20 puts a gain or a loss on every screen that shows a return.

    How test will be performed: The tester turns on the phone's greyscale setting, opens each screen and notification, and records any gain or loss that cannot be told apart from the other without colour.

22. **test-mkt-22: No recommendations**

    Requirements: FR49

    Control: Manual

Initial State: As in test-mkt-20.

Input: The tester opens the same screens and notifications as in test-mkt-20, and makes one trade to bring up the feedback after a trade (FR53). A reviewer then reads every message template in the app and backend, and every item of educational content.

Output: No screen, notification, template, or content item tells the user to buy or sell a named asset, or presents an asset as a "recommended pick".

Test Case Derivation: The fit criterion of FR49 covers every screen and notification. The places where FitFolio names assets in text about the portfolio are the feedback on the change in value (FR52), which names the holdings that changed most, the feedback after a trade (FR53), and the explanation of diversification (FR54). A holds $100/130 = 76.9\%$ of the portfolio value, which is more than `CONCENTRATION_LIMIT` (50%), so the setup brings up all three. The explanation of diversification is about the holding that is too large, which makes it the most likely place for a message such as "sell A". Some messages appear only in conditions that the walkthrough does not create, so the review of every template covers them. Educational content can change after release (FR58), so the review checks the content at the time of the test.

How test will be performed: The tester records any text that tells the user to buy or sell a named asset or presents an asset as a pick. The reviewer does the same for each message template and content item.

23. **test-mkt-23: Trading in a market scenario**

    Requirements: FR45

    Control: Automatic

    Initial State: A test scenario that replays a market downturn, in which the closing price of asset A is 100.00 on its first trading day t_1 and 90.00 on the next, t_2. In the live market, A's current price is 150.00. A test user with a live balance of 500 who holds 2 shares of A and has acknowledged the simulated market notice.

    Input: The user starts the scenario and buys 10 shares of A on day t_1. The user then continues to day t_2, sells the 10 shares, and leaves the scenario.

Output: The purchase leaves a scenario balance of 9,000 and 10 shares, and the sale leaves a scenario balance of 9,900 and no shares. Throughout the scenario and after leaving it, the user's live balance is 500 and they hold 2 shares of A.

Test Case Derivation: FR45 trades in a scenario at the historical prices for the replayed dates. The purchase costs $10 \cdot 100 = 1000$ of the `SCENARIO_START_BALANCE` (10,000), and the sale returns $10 \cdot 90 = 900$, a loss that fits a downturn as in the fit criterion. A purchase at the live price would cost 1,500 and leave 8,500, so the scenario balance shows which price was used. The user holds the same asset in the live market, so a scenario trade that changed the live holding would show. The live balance of 500 is less than the purchase's cost of 1,000, so a purchase that drew on the live balance would be refused. The live balance and holding are checked during the scenario as well as after it, since a change that was undone when the user left would still mix the two portfolios.

How test will be performed: A pytest test starts the scenario, sends each trade and the request to continue, and reads the scenario and live balances and holdings through the API after each step and after leaving.

24. **test-mkt-24: Starting balance of a scenario**

Requirements: FR46

Control: Automatic

Initial State: Two test scenarios, S_1 and S_2, each including asset A at a closing price of 100.00 on its first day. A test user with a live balance of 500 who holds 2 shares of A and has acknowledged the simulated market notice.

Input: The user starts S_1, buys 10 shares of A, and leaves S_1. The user then starts S_2.

Output: On starting S_1, and again on starting S_2, the scenario balance is 10,000 and the user holds no scenario shares.

Test Case Derivation: FR46 starts every scenario with a balance of `SCENARIO_START_BALANCE` (10,000) and no holdings. The user's live balance of 500 and live shares differ from these, so a scenario that copied the live portfolio would start with 500 and 2 shares. The trade in S_1 leaves a balance of 9,000 and 10 shares, so a second scenario that carried on from the first would start with those instead.

How test will be performed: A pytest test starts each scenario, reads its balance and holdings through the API, and sends the trade in S_1.

25. **test-mkt-25: Advancing a scenario**

    Requirements: FR47

    Control: Automatic

    Initial State: A test scenario that starts on Thursday, April 9, 2020, with test historical data for asset A on each trading day. The closing prices are 102.00 on April 9, 98.00 on Monday, April 13, and 101.00 on Tuesday, April 14, and the opening prices on those days are 100.00, 99.00, and 97.50. There is no data for April 10 to 12.

    Input: The user starts the scenario and continues twice.

    Output: On starting, the replayed date is April 9 and A's price is 102.00. After the first continue, the date is April 13 and the price is 98.00. After the second, the date is April 14 and the price is 101.00.

    Test Case Derivation: FR47 moves the replayed date to the next trading day and the prices to that day's closing prices. April 10, 2020, was Good Friday, when US markets were closed, and April 11 and 12 were a weekend, so the next trading day after April 9 is April 13. This catches a scenario that advances by calendar days, or that skips weekends but not market holidays, since both would land on April 10. The second continue is an ordinary step from one weekday to the next. Each day's opening price differs from its closing price, so a scenario that used opening prices would show different prices. The first day is checked as well, since a scenario starts on a replayed day and uses its closing price too.

    How test will be performed: A pytest test starts the scenario, sends each request to continue, and reads the replayed date and A's price through the API after each step.

26. **test-mkt-26: Scenario labels**

Requirements: FR48

Control: Manual

Initial State: A test user signed in on a test device with the release build of FitFolio, with at least one market scenario available.

Input: The tester starts a scenario and opens every screen in FitFolio's navigation that can be reached during it, makes a trade, and continues to the next day. The tester then leaves the scenario and opens every screen of the live market.

Output: Every screen shown during the scenario carries a scenario label and the dates of the period being replayed. No screen of the live market carries a scenario label.

Test Case Derivation: The fit criterion of FR48 covers every screen of a scenario and every screen of the live market. The Hazard Analysis (SAF-15) gives the reason: a replayed crash that looks like the live market could make a user believe it is happening now. Some screens appear only after a trade or after continuing, so the tester does both. The live market is checked after leaving the scenario, which catches a label that stays on after the user leaves. The screens come from the app's navigation, as in test-mkt-20. Scenario prices come from stored historical data, not the market data provider, so this test needs no test backend.

How test will be performed: The tester lists the screens in the app's navigation, opens each one during the scenario and then in the live market, and records any screen whose label or dates are wrong or missing.

27. **test-mkt-27: Removing an asset while its market is closed**

    Requirements: FR31

    Control: Automatic

    Initial State: An asset in the live market whose market is closed, with a last price of 40.00 obtained 2 days ago. Three test users who have acknowledged the simulated market notice. User 1 has a balance of 100 and holds 2 shares. User 2 has a balance of 1,000, holds no shares, and has a queued purchase of 1 share. User 3 has a balance of 0, holds 1 share, and has a queued sale of 1 share.

Input: An administrator removes the asset, and the market then opens. User 1's case is then repeated with the market open at the time of the removal and the same old price.

Output: In both runs, user 1 holds no shares, has a balance of 180, and is told what was sold and at what price. User 2's purchase is rejected, user 2 is told, and their balance is still 1,000. User 3 holds no shares and has a balance of 40, and their queued sale is rejected, and they are told. After the removal, the backend asks the test provider for no more prices for the asset.

Test Case Derivation: FR31 sells at the most recent price whether or not the market is open, so user 1 receives $100 + 2 \cdot 40 = 180$. Waiting for a new price would mean tracking the asset after it is removed, and a delisted asset may never get one. The price is older than `QUOTE_STALENESS_LIMIT` (15 minutes), so the run with the market open checks that FR39 does not refuse the sale, and the run with the market closed checks that FR40 does not queue it. A queued trade would otherwise wait for an opening price that never comes, so user 2's purchase is rejected. User 3 has both a holding and a queued sale, so a balance of 80 would show the share being sold twice. Checking the test provider's requests confirms that the backend stops tracking the asset, which is the reason for selling at removal.

How test will be performed: A pytest test sets up the holdings and queued trades, removes the asset as an administrator, reads each user's balance, holdings, and trades through the API, and checks the requests recorded by the test provider.

4.1.4 Learning and Feedback

These tests cover the learning and feedback requirements in SRS Section 6.4 (FR50 to FR60). The core concepts are the 20 concepts in SRS Table 6.4: ten investing concepts, two Canadian account types, and eight fitness concepts, one for each of the five supported activity types plus active minutes, rest day and streak. Tests of feedback and knowledge check results use test portfolios and a test knowledge check whose values are known, so the expected feedback can be calculated by hand. Prices are sent to the backend directly, as in the

Simulated Market and Portfolio tests, and every test asset's market is open with a price within `QUOTE_STALENESS_LIMIT` (15 minutes).

1. **test-learn-1: Content for every core concept**

   Requirements: FR50, FR55

   Control: Automatic and Manual

   Initial State: The release build of FitFolio on a test device, with its published educational content, and a test user signed in.

   Input: A list of the 20 core concepts in SRS Table 6.4. For each concept, the tester opens its concept explanation from the learning section and starts its knowledge check.

   Output: For every one of the 20 concepts, an explanation opens and a knowledge check starts. A pytest test also finds a published explanation and a published knowledge check for every concept in the list.

   Test Case Derivation: The fit criteria of FR50 and FR55 require an explanation and a knowledge check for every core concept, so the test checks the full list rather than a sample. The automatic check catches a concept with no published content. The manual check catches content that is published but cannot be reached in the app.

   How test will be performed: A pytest test reads the published content through the API and compares it with the list. The tester then works through the list on the device, ticking off each concept and recording any that fail.

2. **test-learn-2: Explanations where concepts appear**

   Requirements: FR51

   Control: Manual

   Initial State: A test user signed in on a test device, holding shares of two test assets, with a streak of at least 1 and at least one completed trade.

   Input: The tester opens every screen in FitFolio's navigation. On each screen, for every core concept shown, the tester opens its explanation and then closes it. On the purchase screen, the tester first enters a quantity of 3.

Output: Every core concept shown on a screen opens its explanation without leaving that screen. After the explanation is closed, the same screen is shown with its state unchanged; on the purchase screen, the quantity is still 3.

Test Case Derivation: The fit criterion of FR51 covers every screen on which a core concept appears. The user holds shares, has a streak and has traded so that the portfolio, activity and trade feedback screens show their concepts, such as portfolio value, return and streak. The quantity on the purchase screen detects an explanation that leaves the screen, since returning to a new copy of the screen would clear it. The screens come from the app's navigation, as in test-mkt-20.

How test will be performed: The tester lists the screens in the app's navigation, records each core concept shown on each one, opens each explanation, and records any that cannot be opened in place.

3. **test-learn-3: Feedback on a change in portfolio value**

   Requirements: FR52, FR60

   Control: Automatic

   Initial State: Three test users, each holding 10 shares of asset A and 10 of asset B. Each last viewed their portfolio when A and B were both priced at 50.00, a portfolio value of 1,000.00.

   Input: Before each user views their portfolio again, the prices change to: user 1, A 56.00 and B 49.00; user 2, A 57.00 and B 49.00; user 3, A 50.00 and B 43.00. Each user then views their portfolio.

   Output: User 1 is shown no feedback on the change. User 2 is shown feedback that names A as the holding that changed most, with A's price change from 50.00 to 57.00 and B's from 50.00 to 49.00. User 3 is shown feedback that names B as the holding that changed most, with B's price change from 50.00 to 43.00.

   Test Case Derivation: `FEEDBACK_THRESHOLD` is 5%, and FR52 gives feedback only when the value has changed by more than it. User 1's value is $10 \cdot 56 + 10 \cdot 49 = 1050$, a change of exactly 5%, so no feedback is shown. User 2's is $570+490=1060$, a change of 6%, and A changed by 70.00 against B's 10.00. User 3's is $500+430=930$, a change of -7%, so a fall also produces feedback, and B changed by 70.00 against A's 0. Taking the largest change by its size rather than its sign makes B the answer for user 3; a calculation that took the largest increase would name A. FR60 (Hazard Analysis, SAF-17) bases feedback on the user's own holdings and price changes and shows them, so the prices in the feedback must match the test prices.

How test will be performed: A pytest test sets each user's holdings, records the view at the starting prices, sends the new prices to the backend, and reads each user's portfolio and feedback through the API.

4. **test-learn-4: Effect of a trade on each holding's share**

   Requirements: FR53

   Control: Automatic

   Initial State: A test user with a balance of 1,000.00, holding 10 shares of asset A and 10 of asset B, priced at 40.00 and 60.00. The prices do not change during the test.

   Input: The user buys 5 shares of A. The user then sells all 15 shares of A.

   Output: After the purchase, the feedback shows A going from 40% to 50% of the portfolio value and B from 60% to 50%. After the sale, it shows A going from 50% to 0% and B from 50% to 100%.

   Test Case Derivation: FR53 shows each holding's share of the portfolio value before and after each trade. Before the purchase, A is $400/1000=40\%$ and B is $600/1000=60\%$. After it, A is $600/1200=50\%$ and B is $600/1200=50\%$. After the sale, A is worth 0 and B is the whole portfolio of 600. The prices give whole percentages, so the expected values need no rounding, and the sale checks a holding that drops to zero.

   How test will be performed: A pytest test sends each trade through the API and checks the before and after shares in each response.

5. **test-learn-5: Explaining diversification**

Requirements: FR54

Control: Automatic

Initial State: A test user with a balance of 500.00, holding 5 shares of asset A and 5 of asset B, both priced at 100.00.

Input: The user views their portfolio. The user then buys 1 share of B and views their portfolio again.

Output: The first view shows no explanation of diversification. The second view shows one, and its text contains none of the symbols or names of the assets on the tradable asset list.

Test Case Derivation: `CONCENTRATION_LIMIT` is 50%, and FR54 explains diversification only when one holding is more than it. Before the purchase, each holding is exactly $500/1000 = 50\%$, so no explanation is shown. After it, B is $600/1100 \approx 54.5\%$, which is above the limit. The fit criterion also requires that the explanation names no asset to buy or sell (Hazard Analysis, SAF-9), and checking the text against the whole asset list covers every asset it could name.

How test will be performed: A pytest test sets up the portfolio, reads the portfolio view before and after the purchase, and searches the explanation's text for each symbol and name on the asset list.

6. **test-learn-6: Results of a knowledge check**

Requirements: FR56

Control: Automatic and Manual

Initial State: A published test knowledge check with three questions, whose correct answers are b, a and c, and whose explanation of question 2's answer is known. A test user signed in.

Input: The user answers b, c and c.

Output: Questions 1 and 3 are marked correct. Question 2 is marked incorrect and shown with the explanation of its correct answer, a.

Test Case Derivation: FR56 marks which answers were correct and explains the correct answer to each question answered wrongly. The answers get the first and last questions right, so the test also catches results that are matched to the wrong question. The explanation is known, so the test can check that the right one is shown.

How test will be performed: A pytest test submits the answers through the API and checks the result. The tester then completes the same check on a test device with the same answers and records what is shown.

7. **test-learn-7: Learning progress**

   Requirements: FR57

   Control: Automatic

   Initial State: A new test user with no learning progress.

   Input: The user opens the explanation of *Diversification*, completes the knowledge check on *Diversification* with every answer correct, and completes the knowledge check on *Stock* with every answer wrong.

   Output: The user's learning progress shows the *Diversification* explanation as viewed and its knowledge check as passed. It does not show the *Stock* knowledge check as passed.

   Test Case Derivation: The fit criterion of FR57 is that a viewed explanation and a passed knowledge check both appear in the user's progress. The SRS does not set a pass mark, so the test uses all answers correct and all answers wrong, which pass and fail under any reasonable pass mark. The failed check shows that progress records only passed checks.

   How test will be performed: A pytest test sends each action through the API and reads the user's learning progress.

8. **test-learn-8: Updating content without an app update**

   Requirements: FR58

   Control: Manual

   Initial State: A test device with a release build of FitFolio installed, and automatic app updates turned off.

   Input: An administrator appends the sentence "Test edit for V&V." to the explanation of *Stock* and publishes a new knowledge check titled "V&V test check". The tester then opens the *Stock* explanation and the list of knowledge checks on the test device.

Output: The explanation of *Stock* ends with the added sentence, and the new knowledge check is in the list. The app's version number is the same as before.

Test Case Derivation: The fit criterion of FR58 is that an edited explanation is shown on a phone whose app has not been updated. FR58 also covers adding content, so the test adds a knowledge check as well. Turning off automatic updates and checking the version number shows that no app update was involved.

How test will be performed: The tester records the app's version number, makes the changes as an administrator, opens the content on the device, and records what is shown and the version number again. The test content is then removed.

9. **test-learn-9: Sources, review dates and second reviewers**

   Requirements: FR59

   Control: Automatic

   Initial State: The published educational content and its content log.

   Input: Every published concept explanation and knowledge check.

   Output: Every explanation has a source and a review date that is not after today. Every explanation and every knowledge check answer key has a reviewer recorded in the content log who is not its author.

   Test Case Derivation: FR59 (Hazard Analysis, SAF-16) shows each explanation with its source and review date, and requires a second team member to review each explanation and answer key before it is published. A reviewer who is the author would not be a second team member, and a review date in the future could not be real.

   How test will be performed: A pytest test reads all published content and the content log, and lists every item that fails a check. It runs on every pull request that changes educational content, so content cannot be published without a second reviewer.

4.1.5 Social Feed

These tests cover the social feed requirements in SRS Section 6.5 (FR61 to FR71). They use four test users, `alex_test`, `bea_test`, `cal_test` and `dev_test`, referred to as A, B, C and D. Unless a test says otherwise, A is friends with B and C and not with D. Automatic tests act as each user through the API with that user's own session, so every check is made from the point of view of the user who would see the result. The times at which updates are shared are set by the backend's test clock.

1. **test-social-1: Sending a friend request**

   Requirements: FR61

   Control: Automatic

   Initial State: A and D are not friends and have no request between them.

   Input: A sends a friend request to the username `dev_test`.

   Output: D's list of received requests contains one pending request from A.

   Test Case Derivation: The fit criterion of FR61 is that entering another user's username sends them a request that they can see. The check is made from D's side, which is where the request must appear.

   How test will be performed: A pytest test sends the request as A and reads D's received requests as D.

2. **test-social-2: Friends only after acceptance**

   Requirements: FR62, FR63

   Control: Automatic

   Initial State: A has no request with C or D. C and D have each shared an update with all their friends.

   Input: A sends a request to D. A then tries to accept that request as A. D declines it. A sends a request to C, and C accepts it.

   Output: While A's request to D is pending, A and D are not friends and A sees none of D's updates. A's attempt to accept their own request is refused, and the request stays pending. After D declines, A and D are still not friends, and the request is no longer in D's list of received requests. After C accepts, A and C are in each other's friend lists.

Test Case Derivation: FR62 makes two users friends only after the user who received the request accepts it, so the sender accepting their own request must not count. FR63 lets the recipient decline, after which the two are not friends and the decliner no longer sees the request. The accepted request to C shows that acceptance does create the friendship, so the refusals are not caused by friendships never forming.

How test will be performed: A pytest test sends each action as the user named in the input and reads the friend lists, received requests and feeds after each step.

3. **test-social-3: Removing a friend**

   Requirements: FR64

   Control: Automatic

   Initial State: A and B are friends. A has shared update U_1 with B, and B has shared update U_2 with A.

   Input: A removes B as a friend.

   Output: U_1 is no longer in B's feed, and U_2 is no longer in A's feed. Neither user is in the other's friend list.

   Test Case Derivation: The fit criterion of FR64 is that after the removal, neither user sees the other's shared updates. Each user has shared one update, so the test checks both directions; removing only the remover's view would leave U_1 in B's feed.

   How test will be performed: A pytest test reads both feeds before and after the removal.

4. **test-social-4: Kinds of update that can be shared**

   Requirements: FR65, FR68

   Control: Automatic and Manual

   Initial State: A has 8,000 steps today, a streak of 12, has reached the 7-day streak milestone, and has passed the knowledge check on *Diversification*. A's profile has an age of 25, a height of 170 cm and a weight of 68 kg, and A has a balance of 450 and a step record of 4,200 steps from 9:00 to 10:00 today.

Input: A shares with B one update of each kind in SRS Table 6.5: their step total for today, their streak, the 7-day streak milestone, and the *Diversification* knowledge check. A then sends requests to share their balance, their portfolio value, their weight, and the 4,200-step record. On a test device, the tester opens A's share screen.

Output: B's feed contains the four updates, showing 8,000 steps, a streak of 12, the 7-day streak milestone and the *Diversification* knowledge check. None of them contains A's age, height or weight, or any time of day. The four other requests are refused and add nothing to B's feed. The share screen offers only the four kinds of update.

Test Case Derivation: The fit criterion of FR65 is that the user can share one update of each kind in SRS Table 6.5 and nothing else, so the test shares each kind and tries four things outside the table. FR68 (Hazard Analysis, SEC-35) excludes body measurements and individual activity records, which the weight and the 4,200-step record test. The Hazard Analysis also excludes the exact time of an activity (SEC-14), which is why no time of day may appear. The device check covers the app, since the requests test only the backend.

How test will be performed: A pytest test sends each share request as A and reads B's feed as B. The tester then records the options on the share screen.

5. **test-social-5: Choosing who sees an update**

   Requirements: FR66

   Control: Automatic

   Initial State: A is friends with B, C and D.

   Input: A shares update U_1 with B only, and update U_2 with B and C. C then requests U_1 by its identifier.

   Output: B's feed contains U_1 and U_2. C's feed contains U_2 only. D's feed contains neither. C's request for U_1 is refused.

   Test Case Derivation: The fit criterion of FR66 is that an update shared with selected friends appears in their feeds and no other friend's. D is a friend who was never selected, which checks that sharing with some friends does not share with all. C's direct request checks that an unselected friend cannot get the update another way (Hazard Analysis, SEC-15).

How test will be performed: A pytest test shares both updates as A, reads each feed as its owner, and sends C's request as C.

6. **test-social-6: Preview before sharing**

   Requirements: FR67

   Control: Manual

   Initial State: Two test devices, one signed in as A and one as B, who are friends. A has activity, a streak, a milestone and a passed knowledge check, as in test-social-4.

   Input: On A's device, the tester starts sharing each of the four kinds of update with B, takes a screenshot of the preview, and confirms. For a fifth update, the tester cancels at the preview.

   Output: A preview is shown before every update is shared. Each update in B's feed shows the same values and text as its preview. The cancelled update does not appear in B's feed.

   Test Case Derivation: The fit criterion of FR67 is that for every update the preview matches what the selected friends see, so each of the four kinds is checked on B's own device. Cancelling at the preview checks that the preview comes before sharing, since an update shared before its preview would already be in B's feed.

   How test will be performed: The tester compares each screenshot with the matching update on B's device and records any difference.

7. **test-social-7: Contents and order of the feed**

   Requirements: FR69

   Control: Automatic

   Initial State: B is friends with A and C and not with D. No updates have been shared.

   Input: A shares U_1 with B at 10:00. C shares U_2 with B at 10:05. A shares U_3 with C only at 10:10. D shares U_4 with all their friends at 10:15. B then reads their feed.

Output: B's feed contains exactly U_2 followed by U_1.

Test Case Derivation: FR69 shows every update shared with the user by a friend, and no others, newest first. U_1 and U_2 are shared with B by friends, so both appear, with U_2 first because it is newer. U_3 is from a friend but not shared with B, and U_4 is from a user who is not B's friend, so neither appears. Each update is shared at a different time, so only one order is correct.

How test will be performed: A pytest test sets the backend's clock to each time, shares each update as its author, and reads B's feed as B.

8. **test-social-8: Deleting a shared update**

   Requirements: FR70

   Control: Automatic

   Initial State: A has shared update U with B and C.

   Input: A deletes U. B then requests U by its identifier.

   Output: U is no longer in B's or C's feed, and B's request for U is refused.

   Test Case Derivation: The fit criterion of FR70 is that a deleted update no longer appears in any friend's feed, so both friends it was shared with are checked. The direct request checks that the update was removed and not only hidden from feeds.

   How test will be performed: A pytest test deletes U as A, reads B's and C's feeds, and sends B's request.

9. **test-social-9: No rankings between users**

   Requirements: FR71

   Control: Manual

   Initial State: Test user A is friends with B, C and D, signed in on a test device. Their totals today are A 20,000 steps, B 12,000, C 1,000 and D 5,000, and their balances follow the same order. C has shared an update with A at 10:00, B at 10:05 and D at 10:10.

Input: The tester opens every screen in FitFolio's navigation, including the friend list and the social feed.

Output: No screen orders users by activity, currency or portfolio performance, or shows a user's position relative to others, such as "1st" or "top". No list of A's friends is in the order B, D, C or C, D, B. The feed lists D's, B's and C's updates in that order.

Test Case Derivation: The fit criterion of FR71 (Hazard Analysis, SAF-11) covers every screen. The test data is chosen so that an ordering by activity or balance can be told apart from every order that is allowed. Sorted by activity, A's friends are B, D, C (or C, D, B from lowest), which differs from their alphabetical order, B, C, D. The feed must be newest first (FR69), which gives D, B, C, so a feed sorted by activity would also stand out. The screens come from the app's navigation, as in test-mkt-20.

How test will be performed: The tester opens each screen, records the order in which users appear on any screen that lists more than one, and records any ranking, position or "top" label.

4.1.6 Reminders and Settings

These tests cover the reminders and settings requirements in SRS Section 6.6 (FR72 to FR77). The reminder types are Activity, Milestone, Portfolio, Learning, and Social, each sent when the event in the SRS's table of reminder types occurs. A user who has not chosen any reminder periods receives reminders during `DEFAULT_REMINDER_PERIOD` (9:00 to 20:00 in their time zone).

Reminders are local notifications that the app creates on the phone, with no push notification service (SRS Section 2.3). The automated reminder tests therefore run against the frontend with Jest, with the phone's clock and its notification service replaced by test versions, so that the time of each event and of each reminder delivered are known. A reminder period includes its start time but not its end time, so two periods that meet, such as 9:00 to 12:00 and 12:00 to 15:00, do not both include 12:00.

1. **test-remind-1: Choosing reminder periods**

   Requirements: FR72

Control: Manual

Initial State: A test user signed in on a test device, who has not chosen any reminder periods.

Input: The tester sets two reminder periods, 7:00 to 8:30 and 18:00 to 19:00, closes FitFolio completely, and opens it again.

Output: The reminder settings show both periods, 7:00 to 8:30 and 18:00 to 19:00.

Test Case Derivation: The fit criterion of FR72 has the user set a period and see it in their settings the next time they open the app, so the app is closed completely, not only sent to the background. FR72 allows more than one period, so two are set, which checks that saving the second does not replace the first. Both periods differ from `DEFAULT_REMINDER_PERIOD`, and the first begins before it, so settings that fell back to the default would be detected.

How test will be performed: The tester sets the periods in the reminder settings, closes the app from the phone's app switcher, opens it again, and checks the periods shown.

2. **test-remind-2: Reminders only in the chosen periods**

   Requirements: FR73

   Control: Automatic

   Initial State: Two test users with every reminder type turned on, in a fixed time zone. User 1 has reminder periods of 7:00 to 8:30 and 18:00 to 19:00. User 2 has not chosen any periods.

   Input: For user 1, friend requests arrive at 6:59, 7:00, 8:29, 8:30, 12:00, and 19:00, and the event of each other reminder type occurs at 12:00. For user 2, friend requests arrive at 8:59, 9:00, 19:59, and 20:00. The clock then runs to the end of the next day.

   Output: User 1's Social reminders are delivered at 7:00, 7:00, 8:29, 18:00, 18:00, and 7:00 the next day, and their other reminders at 18:00. User 2's reminders are delivered at 9:00, 9:00, 19:59, and 9:00 the next day. No reminder is delivered outside the user's periods.

   Test Case Derivation: The fit criterion of FR73 has one user with chosen periods and one without, so there are two users, and user 2's reminders follow `DEFAULT_REMINDER_PERIOD` (9:00 to 20:00). User 1's first period, and user 2's default period, are each tested at both ends, at the boundary and one minute before it. A period includes its start but not its end, so a request at 7:00 or 8:29 is delivered at once, while one at 6:59 or 8:30 is not. The request at 12:00 is between user 1's two periods, and the one at 19:00 is at the end of the second period, so it is delivered the next day. An event outside a period is delivered at the start of the next period, since FR75 requires each event to produce one reminder within the user's periods. Social reminders are used for the boundaries because a friend request is still relevant whenever its reminder arrives, so the expected time depends only on the periods. The other four types each have an event at 12:00, which checks that every type follows the periods, not only Social reminders. Their events are chosen so that they still hold at 18:00: for example, the user records no activity that day, so the Activity reminder is still due.

How test will be performed: A Jest test sets each user's periods, sets the test clock to each time and causes each event, runs the clock to the end of the next day, and records the type and time of every notification sent to the test notification service.

3. **test-remind-3: Turning reminder types off**

   Requirements: FR74

   Control: Automatic

   Initial State: A test user with the default reminder period and every reminder type turned on, in a fixed time zone.

   Input: The test is run five times, once for each reminder type. In each run, that type is turned off, and at 12:00 the event of every reminder type occurs.

   Output: In each run, the user receives one reminder of each of the four types left on, and none of the type turned off.

   Test Case Derivation: The fit criterion of FR74 has one type turned off, which then sends no reminders while the types left on still do. Each type has its own switch, so each is turned off in its own run. The event of every type occurs in every run, so a switch that turned off the wrong type, or more than one type, would show up as a missing reminder of a type left on. The events are at 12:00, inside `DEFAULT_REMINDER_PERIOD`, so the periods cannot be why a reminder is missing.

How test will be performed: As in test-remind-2, with one run for each type.

4. **test-remind-4: Activity reminders**

   Requirements: FR75

   Control: Automatic

   Initial State: Six test users with the default reminder period and every reminder type turned on, in a fixed time zone. Their state on day *d*, all recorded before 9:00, with no activity recorded after it, is:

   - User 1: a goal of 5,000 steps, 2,000 steps, and a streak of 0.
   - User 2: a goal of 5,000 steps, 6,000 steps, and a streak of 0.
   - User 3: as user 1, with day *d* marked as a rest day.
   - User 4: no goal, no activity, and a streak of 3.
   - User 5: no goal, 50 steps, and a streak of 3.
   - User 6: no goal, no activity, and a streak of 0.

   Input: The test clock runs through day *d*.

   Output: Users 1 and 4 each receive exactly one Activity reminder, between 9:00 and 20:00. Users 2, 3, 5, and 6 receive no Activity reminder.

   Test Case Derivation: An Activity reminder is sent when the day is not a rest day, and either the user has not yet met the day's goal or their streak will end if they record no activity (SRS Section 6.6). User 1 has only the first reason, since a streak of 0 cannot end, and user 4 has only the second, since they have no goal. Each user who gets no reminder differs from one of these two in one way: user 2 from user 1 by meeting the goal, user 3 from user 1 by the rest day, user 5 from user 4 by recording activity, which keeps a streak with no goal (test-act-17), and user 6 from user 4 by having no streak to lose. FR75 sends one reminder for each event, so a reminder repeated through the day is also detected.

How test will be performed: A Jest test gives the app each user's goal, totals, streak, and rest days, runs the test clock through day *d*, and records each Activity notification sent to the test notification service.

5. **test-remind-5: Milestone and Portfolio reminders**

   Requirements: FR75

   Control: Automatic

   Initial State: Five test users with the default reminder period and every reminder type turned on, in a fixed time zone, on day *d*. User 1 has a goal of 5,000 steps, no steps yet, and no currency earned. User 2 has no goal and has earned 990 currency in total. Users 1 and 2 have a streak of 0, so no streak milestone can be reached. Users 3, 4, and 5 each hold 1 share of a different asset whose price at the previous day's close was 100.00.

   Input: User 1 records 5,000 steps at 10:00 and 2,000 more at 15:00. User 2 records 1,000 steps at 10:00. At 11:00, the prices of the assets held by users 3, 4, and 5 become 105.00, 105.01, and 94.99.

   Output: Users 1 and 2 each receive exactly one Milestone reminder. Users 4 and 5 each receive one Portfolio reminder, and user 3 receives none. All are delivered between 9:00 and 20:00.

   Test Case Derivation: A Milestone reminder is sent when the user meets their goal or reaches a value in `MILESTONES` (SRS Section 6.6), so there is one user for each. User 1 meets the goal at 10:00, and the steps at 15:00 check that meeting it once gives one reminder, not one for each sync. User 2's 1,000 steps earn 10, which brings their total to 1,000, the first currency value in `MILESTONES`, at a known time. A Portfolio reminder is sent when the portfolio value has moved by more than `FEEDBACK_THRESHOLD` (5%) since the previous day. Each of users 3 to 5 holds one share of one asset, so the change in value is the change in price: exactly 5% for user 3, which is not more than the threshold, and 5.01% for user 4, which is. User 5's price falls by 5.01%, which catches a check that looks only for gains, since the value has moved by more than the threshold in either direction.

   How test will be performed: A Jest test gives the app each event at its time, in the form of the backend's responses, runs the test clock through day *d*, and records each notification sent to the test notification service.

6. **test-remind-6: Learning and Social reminders**

   Requirements: FR75

   Control: Automatic

   Initial State: A test user with the default reminder period and every reminder type turned on, in a fixed time zone, with two friends, F and G.

   Input: On day *d*, an administrator publishes a new concept explanation at 10:00, publishes a new knowledge check at 11:00, and edits an existing concept explanation at 12:00. At 13:00, F shares an update with the user. At 14:00, G shares an update with F only.

   Output: The user receives two Learning reminders, for the new explanation and the new knowledge check, and one Social reminder, for F's update. All are delivered between 9:00 and 20:00.

   Test Case Derivation: A Learning reminder is sent when a new concept explanation or knowledge check is published (SRS Section 6.6), so one of each is published. An edit does not publish new content, so it gives no reminder, which catches a system that reminds every user each time content is corrected. A Social reminder is sent when a friend shares an update with the user or sends a friend request. Friend requests are tested in test-remind-2, so this test uses a shared update. G chooses to share only with F (FR66), so a reminder for G's update would tell the user about an update they cannot see.

   How test will be performed: As in test-remind-5.

7. **test-remind-7: No health data read after consent is withdrawn**

   Requirements: FR76

   Control: Automatic

   Initial State: A test user who has consented to FitFolio reading their health data. The phone's health platform is replaced by a test version that records every read and holds activity records for the current day.

Input: The app syncs. The user then withdraws consent in the app, and new activity records are added to the test health platform. The app is closed and opened again, its background sync runs, and the user opens their activity trends.

Output: The test health platform records reads during the first sync and none after the withdrawal. The app sends no activity totals to the backend after the withdrawal.

Test Case Derivation: The fit criterion of FR76 requires that no further health data be read after the user withdraws consent in the app. Withdrawing consent in the app does not remove FitFolio's permission in the phone's settings, and on iOS the app cannot remove it, so the health platform would still answer reads. The app must stop reading on its own, so the test watches the app's reads, not the permission. The sync before the withdrawal shows that the test platform records reads, so the test cannot pass only because no read was ever recorded. After the withdrawal, the test tries each way the app can read health data: opening the app, the background sync, and the activity screen. The new records give the app something to read.

How test will be performed: A Jest test replaces the health platform and the backend's activity endpoint with test versions, runs the steps, and checks the reads and requests each one records.

8. **test-remind-8: Health permission list**

   Requirements: FR77

   Control: Manual

   Initial State: Two new test users, one on an Android test device and one on an iOS test device, each with the release build of FitFolio.

   Input: On each device, when FitFolio asks for health permissions, the tester allows steps and workouts and denies wheelchair pushes. The tester then opens the health permission list in FitFolio's settings. On Android, the tester then allows wheelchair pushes in Health Connect and opens the list again.

   Output:

- Android: every requested data type is listed with the feature that uses it. Steps and workouts are marked as allowed and wheelchair pushes as not allowed, as in Health Connect. After pushes are allowed, they are marked as allowed.
- iOS: every requested data type is listed with the feature that uses it, with a statement that iOS does not tell apps which types were allowed and where to review them in the Health settings. No type is marked as allowed or not allowed.

Test Case Derivation: FR77's fit criterion has one case for each platform, since Health Connect reports which types an app may read and HealthKit does not (SRS Section 6.6), so there is one device for each. The tester allows some types and denies one, which is the case where the list can be wrong: a list of the requested types shown as allowed would mark pushes as allowed on Android. Pushes are denied because they are the type a user who does not use a wheelchair is most likely to deny. Allowing pushes afterwards checks that the Android list reads the current permissions, not the answers given at setup, since users can change them at any time. On iOS, FitFolio cannot know which types were allowed, so any type marked as allowed or not allowed would be a guess.

How test will be performed: On Android, the tester compares the list with FitFolio's permissions in Health Connect. On iOS, the tester checks the statement and follows the directions to confirm they lead to FitFolio's permissions in the Health settings.

4.2 Tests for Nonfunctional Requirements

[The nonfunctional requirements for accuracy will likely just reference the appropriate functional tests from above. The test cases should mention reporting the relative error for these tests. Not all projects will necessarily have nonfunctional requirements related to accuracy. —SS]

[For some nonfunctional tests, you won't be setting a target threshold for passing the test, but rather describing the experiment you will do to measure the quality for different inputs. For instance, you could measure speed versus the problem size. The output of the test isn't pass/fail, but rather a summary table or graph. —SS]

[Tests related to usability could include conducting a usability test and survey. The survey will be in the Appendix. —SS]

[Static tests, review, inspections, and walkthroughs, will not follow the format for the tests given below. —SS]

[If you introduce static tests in your plan, you need to provide details. How will they be done? In cases like code (or document) walkthroughs, who will be involved? Be specific. —SS]

4.2.1 Area of Testing1

Title for Test

1. test-id1

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input/Condition:

   Output/Result:

   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

   Output:

   How test will be performed:

4.2.2 Area of Testing2

...

4.3 Traceability Between Test Cases and Requirements

[Provide a table that shows which test cases are supporting which requirements. —SS]

5 Unit Test Description

[This section should not be filled in until after the MIS (detailed design document) has been completed. —SS]

[Reference your MIS (detailed design document) and explain your overall philosophy for test case selection. —SS]

[To save space and time, it may be an option to provide less detail in this section. For the unit tests you can potentially layout your testing strategy here. That is, you can explain how tests will be selected for each module. For instance, your test building approach could be test cases for each access program, including one test for normal behaviour and as many tests as needed for edge cases. Rather than create the details of the input and output here, you could point to the unit testing code. For this to work, you code needs to be well-documented, with meaningful names for all of the tests. —SS]

5.1 Unit Testing Scope

[What modules are outside of the scope. If there are modules that are developed by someone else, then you would say here if you aren't planning on verifying them. There may also be modules that are part of your software, but have a lower priority for verification than others. If this is the case, explain your rationale for the ranking of module importance. —SS]

5.2 Tests for Functional Requirements

The detailed design (Module Interface Specification, MIS) has not been written yet, so the unit tests below are organized by the three areas of SRS Section 6 rather than by module. Once the MIS is complete, each area will be mapped to its modules, and each access program of those modules will get at least one test of its normal behaviour and one for each boundary or error case in its specification. The cases listed here are the ones already known from the SRS: boundaries of the symbolic constants, and rules whose failure the system tests above would detect only indirectly. Backend units are tested with pytest in `test/backend`, and app units with Jest, on every pull request.

5.2.1 Accounts and Profile

These tests check the account rules on the backend, below the API, so that a failure points to the rule that caused it rather than to the request.

1. **ut-acct-1: Password length.** A password of 7 characters is refused and one of 8 is accepted, testing `MIN_PASSWORD_LENGTH` (8) on both sides (Hazard Analysis, SEC-38).
2. **ut-acct-2: Age confirmation.** Sign-up details with the age confirmation true are accepted. False and missing are both refused (FR2).
3. **ut-acct-3: Username in use.** Checking `sam_test` when an account already uses it reports it as taken, and checking `jordan_test` reports it as free (FR3).
4. **ut-acct-4: Stored passwords.** After sign-up with the password `Walk2026run`, the stored value is not equal to the password, checking the stored value against the password succeeds, and checking it against `Walk2026ruN` fails (FR4).
5. **ut-acct-5: Reset links.** A reset link works once for the account it was issued to. Using it a second time, or for another account, is refused (FR6).

5.2.2 Learning and Feedback

The feedback rules are calculations on a portfolio, so they are tested as functions with portfolios given directly, with no database or API. Each test has a hand-calculated expected value.

1. **ut-learn-1: Feedback threshold.** With a value of 1,000.00 at the last view, a new value of 1,050.00 (a change of exactly `FEEDBACK_THRESHOLD`, 5%) gives no feedback, and 1,050.01 and 949.99 both give feedback (FR52).
2. **ut-learn-2: Largest change.** For changes of $+70$ in A and -10 in B, A is named. For 0 in A and -70 in B, B is named (FR52).

3. **ut-learn-3: Shares of the portfolio.** For holdings worth 100.00, 100.00 and 100.00, each share is one third, and the shares add up to 100% within `VALUE_TOLERANCE` (0.01). For a single holding, its share is 100% (FR53).
4. **ut-learn-4: Concentration limit.** Holdings worth 500.00 and 500.00 (exactly `CONCENTRATION_LIMIT`, 50%) give no explanation, and 501.00 and 499.00 give one (FR54).
5. **ut-learn-5: Grading.** For a check with answers b, a and c, the answers b, c and c give questions 1 and 3 correct and question 2 incorrect. All correct and all wrong are also tested (FR56).

5.2.3 Social Feed

The visibility rule decides every check in the social feed system tests, so it is tested on its own for every combination of its inputs.

1. **ut-social-1: Visibility.** An update is visible to a viewer exactly when the viewer is its author, or is both a selected viewer and currently a friend of the author. The test checks all four combinations of selected or not and friend or not, and the author (FR64, FR66; Hazard Analysis, SEC-13).
2. **ut-social-2: Friend requests.** A pending request becomes accepted only when its recipient accepts it, and declined when its recipient declines it. Accepting by the sender, and accepting a declined request, are refused (FR62, FR63).
3. **ut-social-3: Building an update.** Each of the four kinds in SRS Table 6.5 builds an update, and any other kind is refused. No built update contains an age, height, weight, individual record, location or time of day (FR65, FR68; Hazard Analysis, SEC-14).
4. **ut-social-4: Feed order.** Updates shared at 10:00, 10:05 and 10:10 are returned newest first, whatever order they are given in (FR69).

5.3 Tests for Nonfunctional Requirements

[If there is a module that needs to be independently assessed for performance, those test cases can go here. In some projects, planning for nonfunctional tests of units will not be that relevant. —SS]

[These tests may involve collecting performance data from previously mentioned functional tests. —SS]

5.3.1 Module ?

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input/Condition:

   Output/Result:

   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

   Output:

   How test will be performed:

5.3.2 Module ?

...

5.4 Traceability Between Test Cases and Modules

[Provide evidence that all modules have been considered. —SS]

References

Author Author. System requirements specification. https://github.com/..., 2019.

John E. Grable and Ruth H. Lytton. Financial risk tolerance revisited: The development of a risk assessment instrument. *Financial Services Review*, 8(3):163–181, 1999.

Team 1, Burn n' Earn. Hazard analysis for fitfolio. https://github.com/davislenover/BurnNEarn/blob/main/pdfs/HazardAnalysis/HazardAnalysis.pdf, 2026a. Version 1.0, October 19, 2026.

Team 1, Burn n' Earn. Software requirements specification for fitfolio. https://github.com/davislenover/BurnNEarn/blob/main/pdfs/SRS-IEEE/SRS.pdf, 2026b. Version 1.1, October 27, 2026.

6 Appendix

This is where you can place additional information.

6.1 Symbolic Parameters

The definition of the test cases will call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance.

6.2 Usability Survey Questions

Participants use FitFolio on their own phones for 4 to 7 days, as part of their normal routine, rather than in one supervised session. The study uses the version of FitFolio from the Revision 0 demonstration and tests its core loop (Development Plan, Section 9): earning virtual currency from steps, buying shares of stocks at real prices, and following the portfolio's value. Shares are bought whole, and steps earn 1 unit of currency per 100 steps (SRS, `EARNING_RATES`), so the stock list used in the study includes at least five stocks priced under 50 USD when the study starts. A participant can then afford a share after about a day of walking. Two of the features being tested span multiple days: earning rewards each day and seeing a portfolio's value change as prices move. A longer study is therefore needed to test them. The study includes at least three days when the stock market is open, so that participants can buy shares and see prices change. The participant profile and the study procedure are given in Section 3.7.

Because the team does not watch participants while they use the app, the evidence comes from what participants record themselves and what they tell the team at the end, in the four parts listed in Table 2. The questionnaires and the log are online forms. All responses, including the notes from each debrief, are recorded under a participant ID rather than a name.

Participants are asked to add to the experience log whenever something works well, goes wrong, or confuses them, rather than at a set time. Each entry is then written while the details are fresh, since they are easily forgotten by the end of the study.

The study ends with a debrief, an interview in which a team member asks the participant about everything they experienced. The interviewer reads the participant's log and questionnaire answers beforehand, so the interview can follow up on specific entries, such as why a screen was confusing or what the

Table 2: Parts of the usability study

Part	When	What it measures
Before-study questionnaire	Day 1, before installing the app (10 minutes)	Background, attitude to risk, and the knowledge check before using the app (G1)
Experience log	Any time, as things happen	What the participant was doing, what worked, what did not, and any help they needed
After-study questionnaire	The day after the last day (10 minutes)	Core tasks done without help (G2, NFR2), larger text (NFR8), and the knowledge check again (G1)
Debrief interview	Right after the questionnaire (30 minutes)	What worked and what did not, and why, and how the participant used their portfolio

participant expected to happen instead. The debrief comes after the after-study questionnaire, so that the conversation cannot change the participant's answers or their score on the knowledge check.

6.2.1 Before-Study Questionnaire

Participants complete this questionnaire on Day 1, after they agree to take part and before they install FitFolio. It has three parts: background, attitude to risk, and the knowledge check for G1. Answers are recorded under the participant ID, and the form asks for no name, contact details, or health measurements.

Part A: Background.

A1. Are you 18 years old or older?

- ○ Yes
- ○ No

A2. Which kind of phone will you use during the study?

- ○ Android
- ○ iPhone

A3. Which statement best describes your experience with investing?

  - ○ I have never invested and know little about it
  - ○ I have never invested, but I know some of the basics
  - ○ I have invested a little, for example in stocks or a fund
  - ○ I invest regularly

A4. Have you used FitFolio before this study?

  - ○ Yes
  - ○ No

A participant must be at least `MIN_USER_AGE` (18) years old (SRS, Appendix C) and must not have used FitFolio before (A4). The team checks these when recruiting, and the form confirms them. The team also checks that the participant's phone runs Android `MIN_ANDROID_VERSION` (17) or iOS `MIN_IOS_VERSION` (27) or later (SRS, SC1). A2 records which kind of phone each result comes from.

Most participants are beginners at investing, which means any answer to A3 except "I invest regularly". One or two participants who invest regularly also take part, since their experience gives useful feedback. They do the same study, but their results are reported separately and do not count toward G1, G2, or NFR2.

Part B: Attitude to risk. How willing a participant is to take risks is likely to shape how they use their portfolio. People often describe themselves differently from how they act, so these questions describe everyday situations instead of asking about risk directly. They were written for this study, based on the kinds of questions in a published risk tolerance scale (Grable and Lytton, 1999). The Usability Report compares these answers with what the participant shows and tells the interviewer in the debrief (I7).

On the form, this part is titled "Your money choices", so that the title does not say what is being measured. Participants see this instruction before the questions:

> There are no right or wrong answers. For each question, choose what you would most likely do.

B1. Imagine you have saved $1,000 that you will not need for five years. You can keep it in a savings account, which pays a small amount of guaranteed interest, or invest it in something such as stocks, whose value can rise or fall. What would you do with it?

- ○ Keep all of it in the savings account
- ○ Keep most of it in the savings account and invest the rest
- ○ Invest most of it and keep the rest in the savings account
- ○ Invest all of it

B2. Imagine you invested $1,000 in the stock market, and a month later it is worth $800. What would you most likely do?

- ○ Sell all of it
- ○ Sell some of it
- ○ Keep all of it and wait
- ○ Keep all of it and invest more

B3. You can put $1,000 into one of these for a year. Which would you choose?

- ○ One that will be worth between $1,000 and $1,050
- ○ One that will be worth between $900 and $1,200
- ○ One that will be worth between $600 and $1,600

Part C: Knowledge check. Participants see this instruction before the questions:

> This is not a test of you. Please answer without looking anything up, and choose "I don't know" rather than guessing. We will go through the answers with you at the end of the study.

C1. What does it mean to own one share of a company's stock?

(a) You own a small part of the company.
(b) You have lent the company money, and it will pay you interest.
(c) You can buy the company's products at a discount.

C2. What is a stock's market price?

(a) The price the company set when it first sold its shares.
(b) The price at which one share was last bought and sold.
(c) The average price of the stock over the past year.

C3. You bought 3 shares of one stock at \$20 each and 2 shares of another stock at \$50 each. The first stock is now priced at \$30 and the second at \$40. What is your portfolio worth now?

(a) \$70
(b) \$160
(c) \$170

C4. You bought one share for \$50, and it is now worth \$60. What is your return?

(a) 10%
(b) 20%
(c) 120%

C5. Why might an investor spread their money across many different investments?

(a) So that they are sure not to lose any money.
(b) So that if one investment drops, the total drops less.
(c) So that they pay less tax on what their investments earn.

C6. The price of Stock A often rises or falls by 5% in a day. The price of Stock B rarely moves more than 1% in a day. Which statement is true?

(a) Stock A is more volatile, so its value is harder to predict in the short term.
(b) Stock A is more volatile, so it will earn more than Stock B over time.
(c) Stock B is more volatile, so its value is harder to predict in the short term.

The questions cover the investing concepts from SRS Section 6.4 that the core loop involves. The core loop teaches no fitness concepts, so none are asked. Each has one correct answer (Table 3), and the form also offers "I don't know". The after-study questionnaire asks the same questions again. A correct answer scores 1, "I don't know" scores 0, and a wrong answer scores minus one half, so guessing does not raise a score on average. A participant improves if their score goes up by at least 1 point, and G1 is met if at least 70% of beginners improve, apart from those left out in Part F.

Table 3: Knowledge check answers

Question	**Core concept**	**Answer**
C1	Stock and share	(a)
C2	Market price	(b)
C3	Portfolio value	(c)
C4	Return	(b)
C5	Diversification	(b)
C6	Risk and volatility	(a)

6.2.2 Experience Log

On Day 1, participants save a link to the log on their phone's home screen, so that adding an entry takes about a minute. Each entry is a short form with the questions below. The link fills in the participant's ID, and the form records the date and time on its own.

L1. Which part of FitFolio were you using?

  ○ Letting FitFolio read my steps
  ○ My currency or rewards
  ○ Buying shares
  ○ My portfolio
  ○ Something else, such as installing the app

L2. How did it go?

  - ○ It went well
  - ○ It worked, but it was hard or confusing
  - ○ It did not work

L3. What were you trying to do, and what happened? (Written answer)

L4. Did you need help from outside the app?

  - ○ No
  - ○ Yes, I searched online
  - ○ Yes, I asked someone
  - ○ Yes, something else

The first four choices in L1 match the core tasks D1 to D4, so the Usability Report can group entries by task. L4 shows where a participant needed help from outside the app.

6.2.3 After-Study Questionnaire

Participants complete this questionnaire the day after the last day of the study, just before the debrief. It has no written questions, since the debrief covers what worked and what did not. Its parts continue from the before-study questionnaire.

Part D: Core tasks. These four tasks, which follow the core loop in order, are the core tasks for this study. Participants are given them on Day 1 as things to try during the study. For each task, participants choose one of these answers:

- ○ Yes, without help from outside the app
- ○ Yes, but I needed help from outside the app at least once
- ○ No, I tried but could not
- ○ No, I did not try

D1. Let FitFolio read your steps.

D2. See the currency you earned from your steps.

D3. Buy shares of a stock.

D4. Check how much your portfolio is worth.

A participant completes the core tasks if they choose the first answer for all four, and none of their log entries shows outside help with one of them. Help from the team with installing the app, or with a problem on the team's side such as the server being down, is not outside help. G2 and NFR2 are met if at least `MIN_TASK_SUCCESS_RATE` (80%) of beginners complete the core tasks.

Part E: Larger text. On one weekday of the study, the team sends each participant the steps for their kind of phone to set its text size to the setting closest to `MAX_TEXT_SCALE` (200%) of the default. It asks them to buy at least one share and check their portfolio that day.

E1. On the day you used larger text, could you still read everything and use every button in FitFolio?

- ○ Yes
- ○ Mostly, but some text or buttons were cut off or hidden
- ○ No, I could not use some screens
- ○ I did not use larger text

NFR8 is met if at least one participant on each kind of phone used larger text, and no participant reports a problem on a screen used for a core task.

Part F: Knowledge check again. Participants answer C1 to C6 again, with the same instruction as before, and then one more question:

F1. During the study, did you look up the answer to any of these questions, or ask anyone about them?

- ○ Yes
- ○ No

A participant who answers Yes, or who had full marks before the study, is left out of the G1 results.

6.2.4 Debrief Interview

The debrief takes about 30 minutes, in person or by video call. One team member asks the questions and a second takes notes. The interviewer asks open questions and does not explain how the app works until the end, so that the answers describe the participant's own experience. A question that starts with a condition in italics is asked only when the condition applies, and is skipped for anything the participant has already explained.

- I1. Walk me through how you used FitFolio during the study.
- I2. What worked well for you, and why?
- I3. What was the most confusing or frustrating moment? What did you expect to happen instead?
- I4. *For each log entry where L2 was not "It went well" or L4 was not "No":* You wrote that … Can you tell me more about that?
- I5. *For each Part D task not answered "Yes, without help from outside the app":* What happened with this task?
- I6. *If E1 was not "Yes" or "I did not use larger text":* Which screens had the problem, and what happened on them?
- I7. Can you show me your portfolio? How did you decide what to do with your currency, and what, if anything, did you do as prices moved?
- I8. What, if anything, did you learn from using FitFolio?
- I9. *If A3 was "I invest regularly":* How does FitFolio compare with how you invest now, and what do you think a beginner would find hard?
- I10. If you could change one thing about FitFolio, what would it be?

After I10, the interviewer goes through the knowledge check answers with the participant and answers their questions. The Usability Report groups the notes by step of the core loop, in the same way as the log entries.

7 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

7.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

7.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

7.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

8 Appendix — Reflection

[This section is not required for CAS 741 —SS]

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

Davis: My main pain point was how much ground this deliverable had to cover, and a lot of that came from our own earlier work. The SRS has 77 functional requirements, and each one needs at least one test. We rushed to get the SRS done and did not really check whether all of those requirements were needed, so that cost came back here. My share of Section 4.1 alone came to 57 tests. I resolved this by splitting 4.1 with Farid, using one test format across the whole section, and working through it one feature area at a time. The usability questionnaire had a similar problem. It grew long and elaborate, and I could see that justifying all of it would be hard. I resolved this by cutting it back to what each goal actually needs and limiting the study to the core loop. Next time I will check requirements more carefully before moving on, since every requirement we keep has to be tested later.

Farid: My first pain point was coordinating with Davis on Section 4.1. We split the section by feature area, but I branched from `main` before his tests were merged, and we had both replaced the same part of the template. The result was a merge conflict, and my tests had no introduction above them, since the conventions they depended on were on his branch. I resolved it by merging his work into my branch and placing my areas around his in the order of SRS Section 6. Next time, when two of us write parts of the same section, I will branch from the teammate's branch or agree first on who writes the shared text. The second pain point was that writing tests with exact inputs and expected outputs exposed gaps in requirements I had written myself. FR57 records a passed knowledge check but never says what a pass is, FR3 does not say whether usernames are case-sensitive, and FR52 does not say what feedback to give when the previous portfolio value was zero. I could not write an exact expected output for any of these. Where I could, I designed around the gap, for example by using all-correct and all-wrong answers for FR57, and I raised each gap as an issue so the SRS gets fixed instead of the test hiding it. This showed me that a requirement with a fit criterion can still be impossible to test, and that writing the test is a good way to find out.

Zaid: My main difficulty was understanding what belonged in Implementation Verification and how it differed from the Development Plan and Section 3.6. These sections discuss similar topics, so I was unsure how much detail to include without repeating information. The distinction became clearer when I focused on how the team would check the code, while leaving the workflow details in the Development Plan and the tool descriptions in Section 3.6. This helped me explain what reviewers would check, how tests would check the requirements, and how problems would be recorded. I also found that small wording choices could make the plan unclear. For example, saying that expected results would be calculated "independently from the SRS rules" could suggest that the rules should not be used. The intended meaning was to use the SRS to calculate the expected answer independently of the code being tested. Revising that sentence helped me understand why a testing plan needs precise wording. Next time, I will identify the related sections before drafting and check that each statement clearly explains what the team will do.

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?